BURGUERCLICKER



Javier Paz Ceballos

Y

Andrés Galindo Escalante

2º Grado superior de Desarrollo de Aplicaciones Multiplataforma

Tutor: Sergio Fontán Llama

RESUMEN

El proyecto es un juego llamado “BurgerClicker”, el objetivo es ir ganando el mayor peso posible haciendo clicks en la hamburguesa, o de forma pasiva comprando los distintos ingredientes.

A medida que se va desarrollando la partida se va incrementando cada vez más el peso total, se adquieren nuevas tiendas, y se desbloquearan los distintos logros que incluye el juego.

Las distintas tiendas que incluye el juego no solo servirán para ir ganando peso de forma pasiva, sino que también irán añadiendo los ingredientes comprados a la hamburguesa principal.

Otra mecánica del juego consiste en desbloquear distintas mejoras, las cuales ayudan al usuario otorgándole beneficios, por ejemplo, una de estas mejoras duplica el valor de los clicks.

Cada usuario podrá tener hasta 3 partidas diferentes en su dispositivo, en las cuales el progreso será diferente entre ellas.

ABSTRACT

The project is a game called “BurgerClicker” the objective of the game is to win as much weight as possible, by clicking the burger, o passively by buying the different ingredients.

As the game progresses the weight increases exponentially, the player acquires new shops, and they unlock the different achievements included in the game.

As the player buys the different upgrades included in the game, they not only serve to gain passive weight, but they also add new ingredients to the main burger.

There is also another mechanic in the way of upgrades, which consists in unlocking different upgrades, which help the player by granting him numerous benefits, like doubling the gains per click.

Each user can create up to three different games in his device, each with its own progression.

INDICE

BURGUERCLICKER 0
RESUMEN 1
ABSTRACT 2
JUSTIFICACIÓN 4
INTRODUCCIÓN 5
ANALISIS DE MERCADO 6
OBJETIVOS QUE SE BUSCA CONSEGUIR 7
MARCO TEÓRICO 8
Tecnologías 8
Metodología 11
Diagrama de base de datos 13
Diagrama de interfaces 14
Flujo de interfaces 17
Diagrama de clases 18
Requisitos, funcionalidades, tareas, pruebas 23
Casos de uso 32
Pruebas 41
Diagrama de Gantt 44
PLANES A FUTURO 48
GIT 49
Enlace al proyecto 49
Capturas 49
BIBLIOGRAFIA 51

JUSTIFICACIÓN

Los videojuegos son una forma de entretenimiento apreciada por millones de personas en todo el mundo, incluidos los creadores de este proyecto, razón por la cual se decidió hacer uno como proyecto.

Se decidió hacer un videojuego basado en el género de los “Idle Clicker”, según (King, 2015) es un género bastante adictivo y llamativo. Este tipo de juegos se basan en ir ganando un recurso de manera incremental, ya sea dinero, galletas o en el caso de este proyecto, peso. Este recurso se suele obtener en cantidades limitadas al principio del juego, normalmente pulsando un objeto, pero a medida que se progresa se desbloquean nuevas maneras de ir consiguiendo cantidades cada vez más grandes del recurso, mediante tiendas u otras mejoras, las cuales normalmente aportan el recurso de forma pasiva, lo que otorga el nombre a este género.

Suelen tener una serie de logros que se van desbloqueando a medida que progresa el juego y el jugador llega a ciertas metas. Estos juegos no suelen tener final, sino que el objetivo es conseguir el máximo posible del recurso, y obtener todos sus logros.

El poder ir consiguiendo pequeñas mejoras a corto plazo dentro del juego hace que el jugador siempre este tentado a seguir jugando un rato mas para poder ir desbloqueando la siguiente mejora.

Este proyecto toma prestados conceptos de este género, pero aporta su propio enfoque y diseño únicos, además de dar un toque humorístico, al no ser un concepto demasiado serio.

INTRODUCCIÓN

La aplicación se centra en la creación de un juego para plataformas móviles, llamado "BurgerClicker", diseñado para dispositivos con sistema operativo Android.

Como se indica en la justificación, el juego se basa en el género "Idle Clicker", donde los jugadores deberán aumentar su peso, ya sea al pulsar la hamburguesa o de forma pasiva a través de la adquisición de las distintas tiendas y mejoras.

Es interesante desarrollar un juego de este estilo, ya que, aunque parezca simple tiene una compleja lógica detrás de él, principalmente a la hora de calcular precios y costes, además de la obtención de peso por parte del usuario.

El sistema de mejoras ayuda al usuario a ganar más peso, aumentando las ganancias por clic, o incrementando el aporte de las tiendas, y otorga al usuario una sensación de progreso.

Este concepto de juego ha sido elegido debido a la sencillez y adicción que han impulsado el éxito de los Idle Clickers, donde el progreso avanza de manera constante sin exigir una incesante vigilancia.

Esta memoria detalla el funcionamiento del juego, que pasos se han realizado para desarrollarlo, que pruebas se han hecho para asegurar el correcto funcionamiento del juego, y que tecnologías se han empleado.

ANALISIS DE MERCADO

La situación actual del mercado es bastante competitiva, porque hay bastantes juegos muy similares a “Burger Clicker”, como por ejemplo "Cookie Clicker” o “Clicker Heroes”. Esto a su vez supone una oportunidad, ya que existe una amplia garantía de clientes potenciales, los cuales estén familiarizados con este tipo de juegos, disfruten con ellos y estén dispuestos a invertir dinero en ellos.

Lo que diferenciará a este juego de todos los existentes, es que, al ser una aplicación sencilla de entender y jugar, no requerirá que el usuario invierta gran parte de su tiempo. Incluso si se siente la necesidad de mejorar en el juego, puede dedicar intervalos de 10 minutos aproximadamente y con eso ira mejorando en el juego sin dificultad de forma pasiva y constante.

El canal principal de distribución de la aplicación sería la tienda “Play Store”, utilizada por todos los dispositivos Android, lo cual supondría un pequeño coste.

OBJETIVOS QUE SE BUSCA CONSEGUIR

El principal objetivo que se busca con el proyecto es crear una aplicación fácil e intuitiva de usar, para que pueda ser utilizada en cualquier momento del día como fuente de distracción y entretenimiento.

La intención es hacer esta aplicación adictiva para sus usuarios manteniendo un funcionamiento sencillo, e implementar mejoras progresivas para crear al usuario la necesidad de jugar todos los días para poder llegar a completar todos los objetivos del juego.

Una aplicación con objetivos similares sería Tik Tok, la cual también es fácil de usar y muy adictiva, utilizada por millones de usuarios para distraerse fácilmente en cualquier momento del día.

MARCO TEÓRICO

Tecnologías

El entorno empleado para desarrollar el juego es "Android Studio", ya que el juego está dirigido a dispositivos móviles con sistema operativo Android, en concreto la versión "Android 14 (API level 34)".

Es un entorno bastante sólido, que otorga muchas facilidades para desarrollar aplicaciones Android, como una interfaz gráfica para desarrollar las vistas, herramientas de ayuda de código, y un gran número de plantillas para desarrollar diversos proyectos. Además, incluye robustos emuladores para poder probar la aplicación como si se tratará de un dispositivo Android verdadero. También ofrece un potente modo de depuración, que ayuda enormemente a encontrar fallos en la aplicación.

Los desarrolladores del proyecto ya tenían experiencia previa con este entorno, lo cual contribuyo a la decisión de emplearlo.

Kotlin es el lenguaje empleado para el desarrollo del "Backend" de la aplicación, ya que es el lenguaje que mejor se adapta a Android, ofreciendo muchas facilidades para crear aplicaciones Android, existiendo un gran número de librerías y funcionalidades centradas en ello.

Según (Team, 2021) estas son algunas de las razones principales para usar Kotlin:

- Es un lenguaje conciso, y más rápido que Java, que sería la principal alternativa.
- Obliga a manejar los null, lo que evita fallos críticos en los programas, algo que Java no hace.
- Se puede convertir de Kotlin a Java dentro del propio Android Studio, de manera automática, algo bastante útil.

Es un lenguaje que resulta bastante fácil de aprender si se parte de Java, o cualquier otro lenguaje orientado a objetos.

La base de datos empleada es Firebase Realtime Database (FireBase, s.f.), que se adapta bastante bien a las necesidades del proyecto, aportando una gran facilidad y rapidez para cambiar y acceder a los datos en tiempo real, requerimiento para el funcionamiento del juego.

Es relativamente sencilla de emplear, aunque hay que tener en cuenta que funciona de forma asíncrona, lo que puede causar algún problema si no se tiene en consideración; pero generalmente es bastante fiable y rápido. La base es de naturaleza no relacional, y los datos se guardan en formato JSON, lo que facilita la compatibilidad.

Para los iconos de las mejoras de la aplicación se usaron varias páginas, que se listan a continuación:

(Icon Library, s.f.)

(Vecteezy, s.f.)

Para editar las imágenes se ha usado Adobe Photoshop, además del editor web:

(Photopea, s.f.). El primero es un software de pago, con bastantes funcionalidades y gran capacidad de edición de imágenes, muy popular para el diseño gráfico, la única gran desventaja siendo su elevado precio. Photopea es una página web gratuita que tiene muchas de las funcionalidades de Adobe Photoshop, aunque no implementa algunas de las más avanzadas. Aun así, es una herramienta bastante útil para la edición de imágenes, y para los propósitos de este proyecto es más que suficiente.

Además, se han tenido que optimar las imágenes de la aplicación debido a su elevado coste en el rendimiento de la aplicación, para ello se ha utilizado el siguiente sitio web (Optimizilla, s.f.).

Los sonidos empleados se han obtenido de las siguientes páginas, que ofrecen una gran variedad de sonidos de uso libre:

(PixaBay, s.f.)

(soundbuttonsworld, s.f.)

El programa empleado para editar los sonidos ha sido ShotCut, que no solo es fácil de usar y contiene un gran número de funcionalidades, sino que también es completamente gratuito y de código abierto.

Para diseñar el diagrama de base de datos, se ha empleado el siguiente sitio web (JSONCrack, s.f.). Esta página ofrece la posibilidad de visualizar un documento JSON de forma gráfica, lo cual es beneficioso ya que la base de datos de Firebase es un JSON.

El diagrama de flujo de interfaces se ha llevado a cabo mediante el siguiente dominio web (Lucidspark, s.f.). Este sitio web cuenta con un editor esquemático bastante potente, gracias a todas las funcionalidades que brinda dicho editor web se ha podido realizar un diagrama bastante claro y visualmente atractivo.

A fin de mantener un control de versiones se ha optado por emplear el siguiente servicio web (GitHub, s.f.). Es un servicio gratuito el cual permite tener almacenadas las distintas versiones de un proyecto de código, y permite que varias personas puedan colaborar en el mismo proyecto subiendo cada una su versión pertinente.

Metodología

Para la organización, el buen funcionamiento y productividad de la aplicación, se decidió seguir parte de la metodología Scrum.

Para ello se plantearon distintas reuniones, siempre los sábados, en las cuales se comentaba el progreso que se iba teniendo sobre la aplicación, así como los problemas e inconvenientes que surgieran durante la semana.

También al final de estas reuniones se asignaba un sprint a cada persona, estos estaban compuestos por pequeñas tareas con una baja carga de trabajo, para poder llevarlos a cabo durante la semana sin complicaciones y tenerlos listos para el fin de semana siguiente.

Gracias a esta forma de organización se han podido llevar a cabo todas las tareas a tiempo y así poder presentarlas correctamente en las distintas entregas que se han realizado sobre el proyecto.

Otra metodología que se optó por utilizar en el proyecto es la metodología en cascada o waterfall.

Para ello se empezó planificando de que iba a tratar la aplicación, y se decidió que iba a ser un juego para dispositivos móviles de la categoría “Idle Clicker”.

Posteriormente se planifico el diseño de la aplicación, optando por un enfoque colorido y llamativo, por ejemplo, se han utilizado fondos de distinto color para cada apartado de la aplicación

Al terminar con el diseño de la aplicación se procedió a planificar la base de datos y la estructura del proyecto.

Para la base de datos se decidió organizar los distintos recursos en diferentes objetos y para el proyecto se pensó en estructurarlo utilizando el patrón de arquitectura Modelo-Vista-Controlador (MVC).

Finalmente, una vez ya estaba todo desarrollado, se realizó una fase de verificación en la cual se hicieron pruebas en toda la aplicación para comprobar que no hubiera ningún error durante la experiencia de juego.

Diagrama de base de datos

Ya que Firebase se trata de una base de datos no relacional, no se pueden mostrar las relaciones con un esquema convencional, por lo que se ha utilizado la web (JSONCrack, s.f.), para realizarlo, mostrando solo un ejemplo de cada logro, mejora y tienda para mejor visualización.

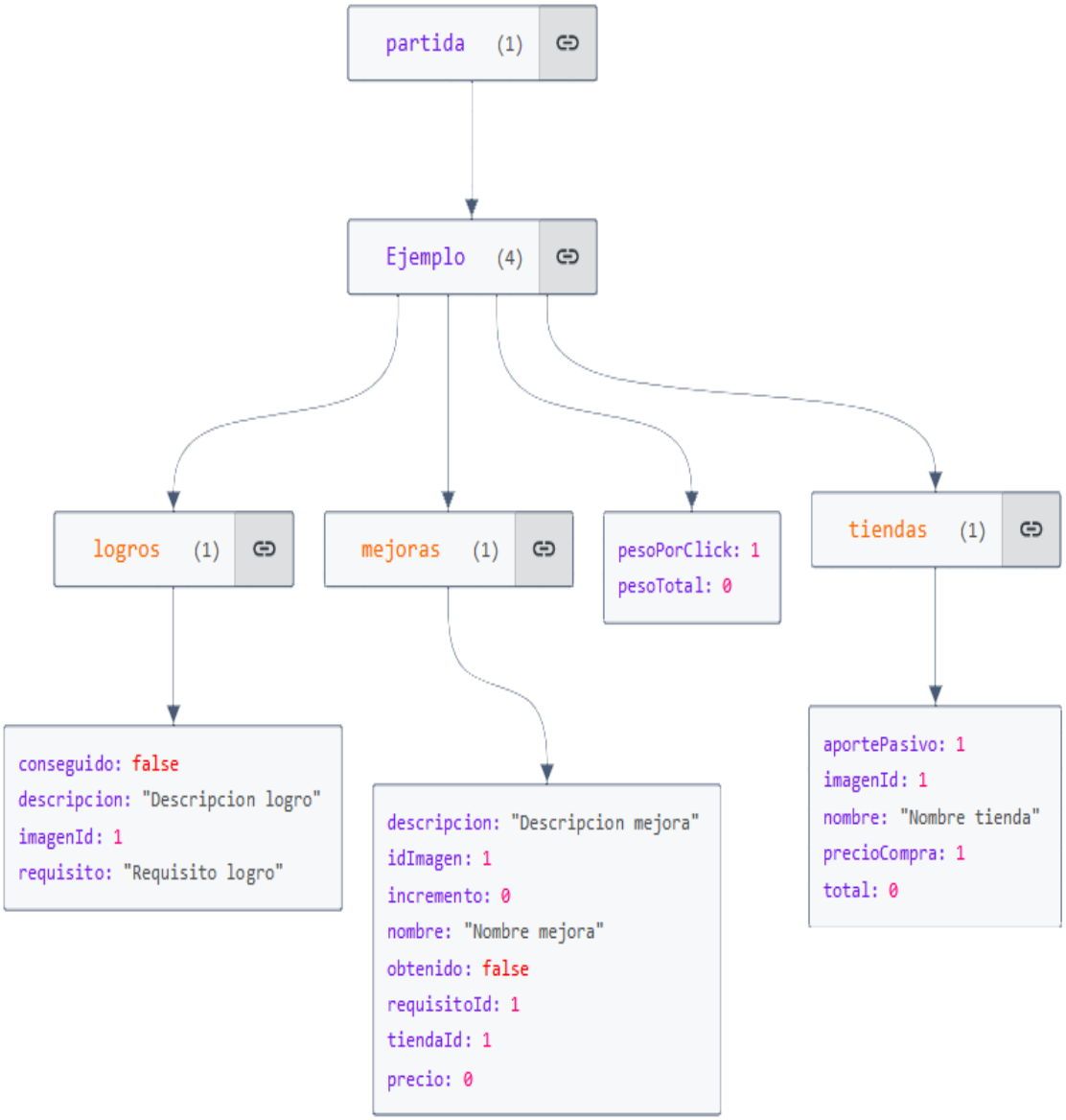
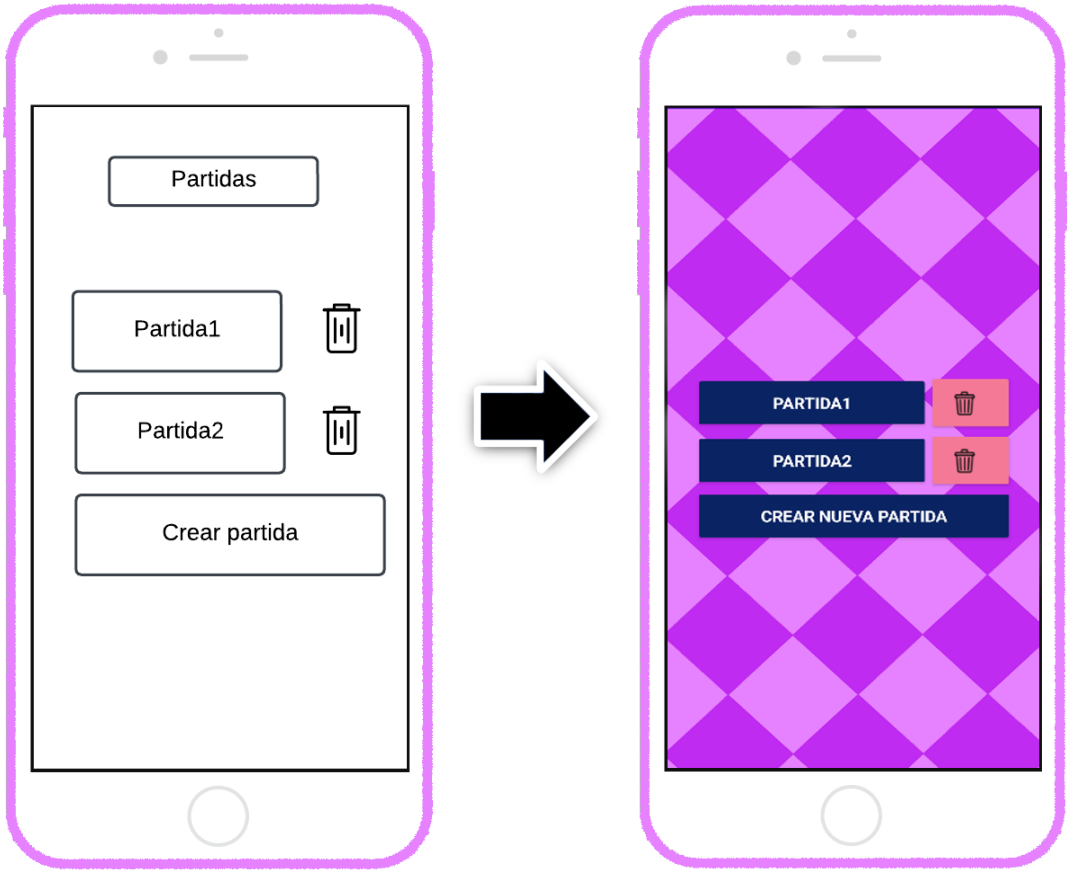
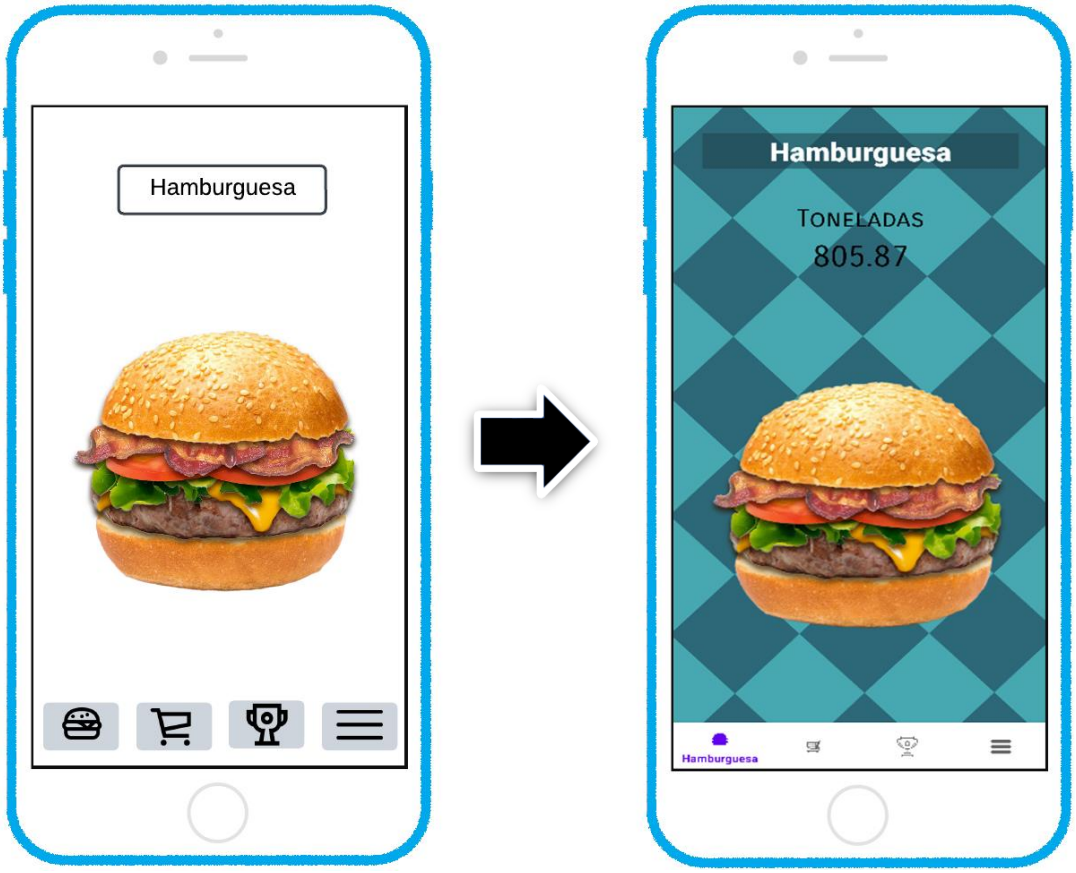
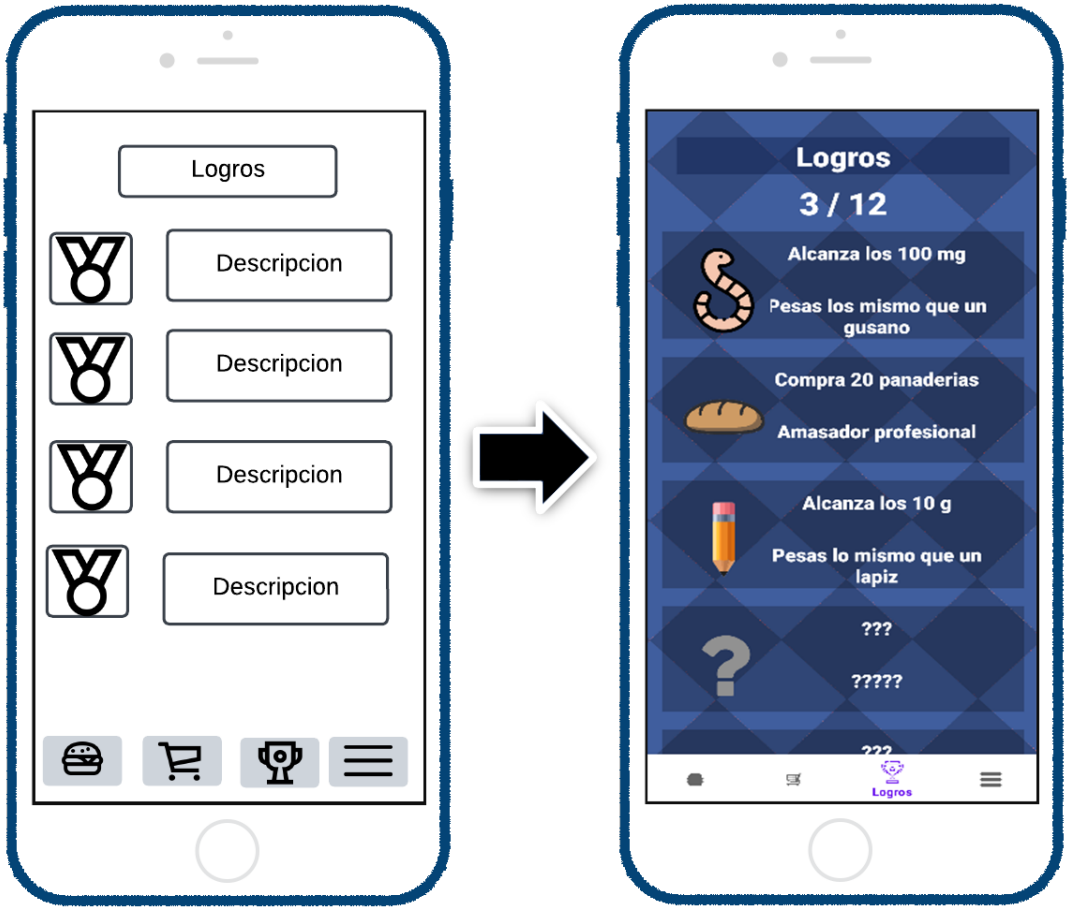


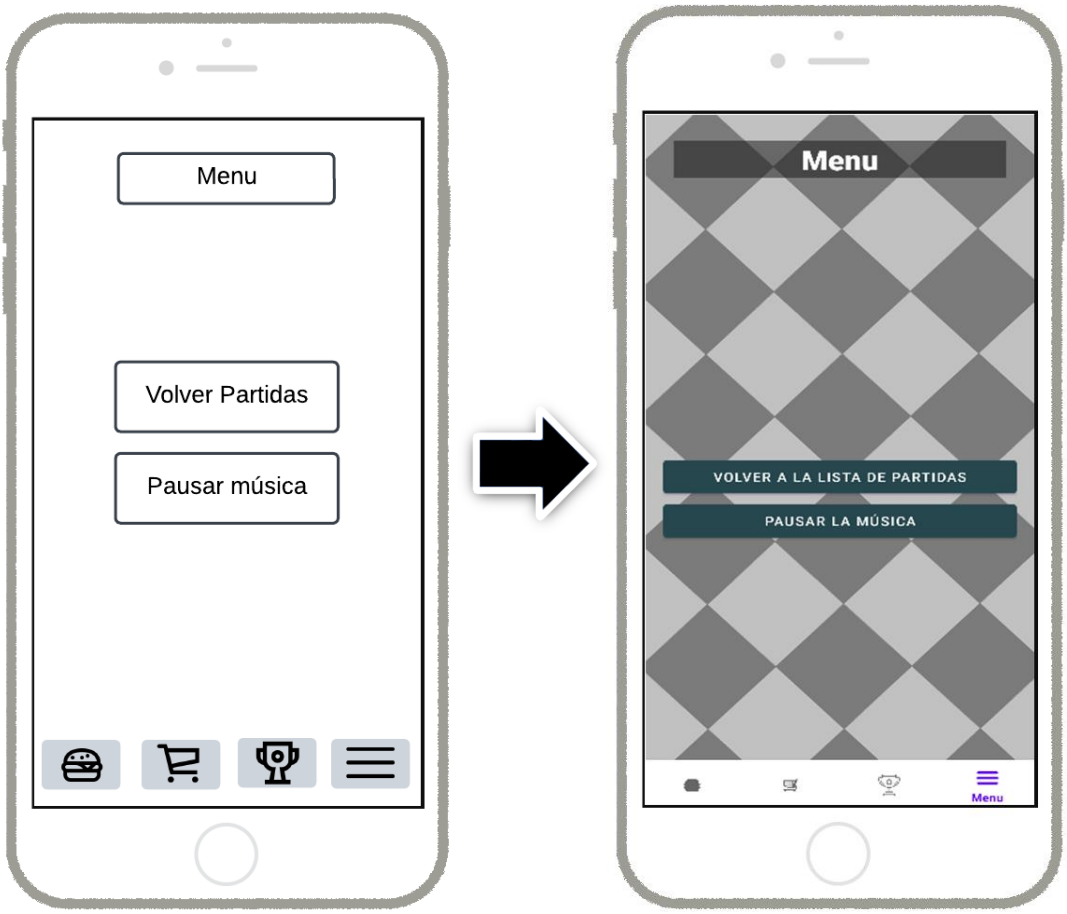
Diagrama de interfaces

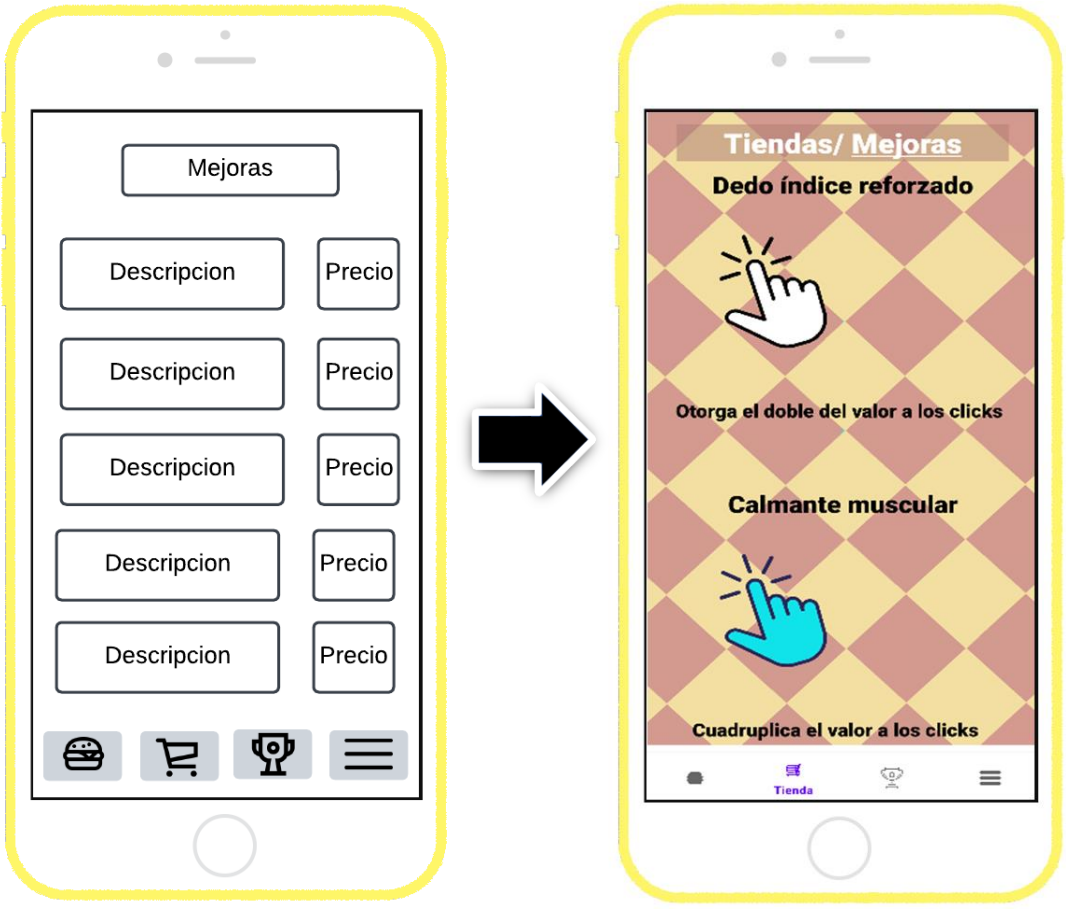





Tiendas
Mejoras
Total
Precio
Comprar
Recompensa por segundo
Total
Precio
Comprar
Recompensa por segundo
Tiendas / Mejoras
1
150.0 Mg
COMPRAR PANADERIA
7.5 Mg por segundo
1
1.0 G
COMPRAR CARNICERIA
50.0 Mg por segundo
Tienda


Logros
Descripcion
Descripcion
Descripcion
Descripcion
Logros
3 / 12
Alcanza los 100 mg
Pesas los mismo que un gusano
Compra 20 panaderias
Amasador profesional
Alcanza los 10 g
Pesas lo mismo que un lapiz
???
?????
Logros


Menu
Volver Partidas
Pausar música
Menu
VOLVER A LA LISTA DE PARTIDAS
PAUSAR LA MÚSICA
Menu


Mejoras
Descripcion
Precio
Descripcion
Precio
Descripcion
Precio
Descripcion
Precio
Descripcion
Precio
Tiendas/ Mejoras
Dedo índice reforzado
Otorga el doble del valor a los clicks
Calmante muscular
Cuadruplica el valor a los clicks
Tienda

Flujo de interfaces

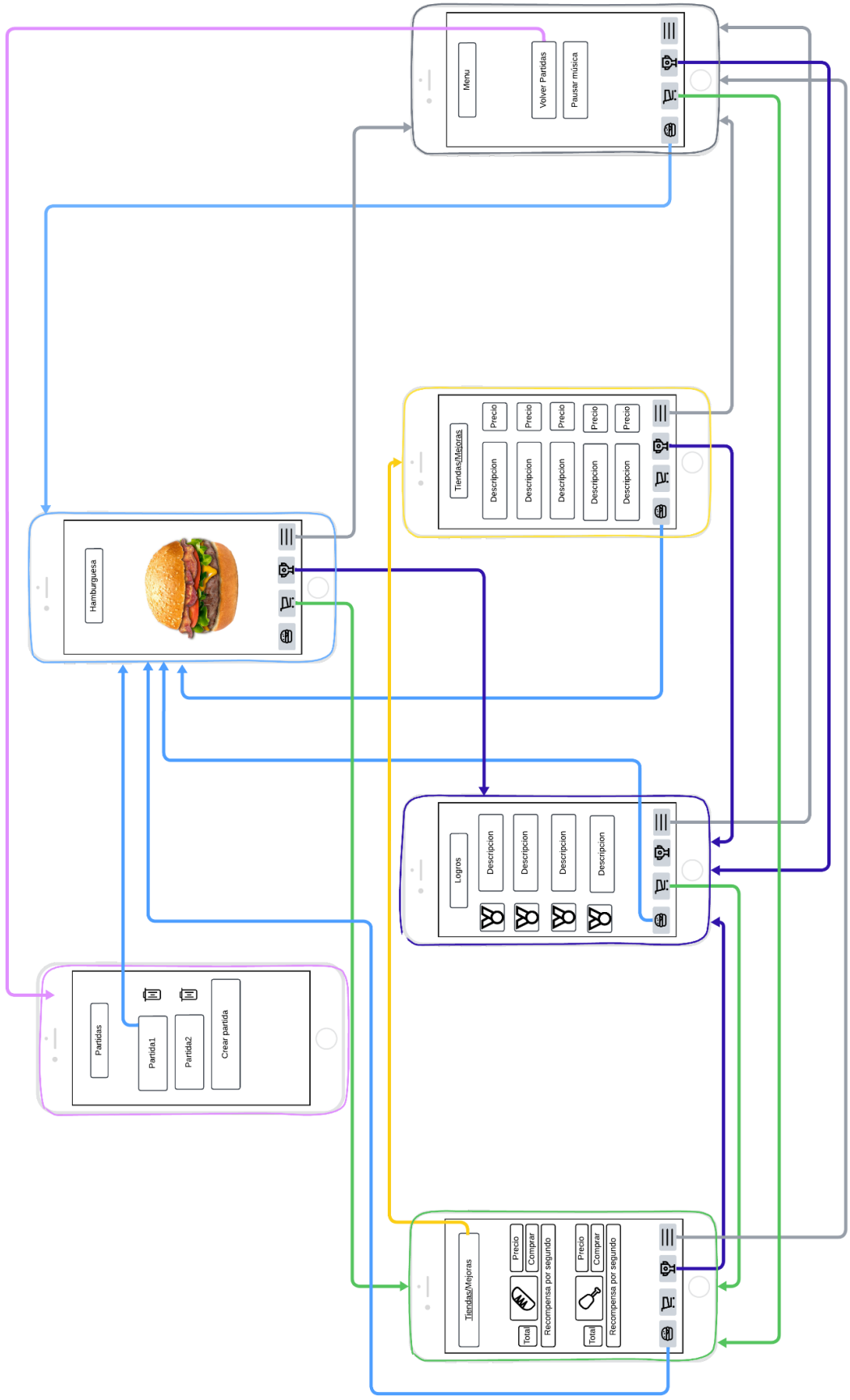
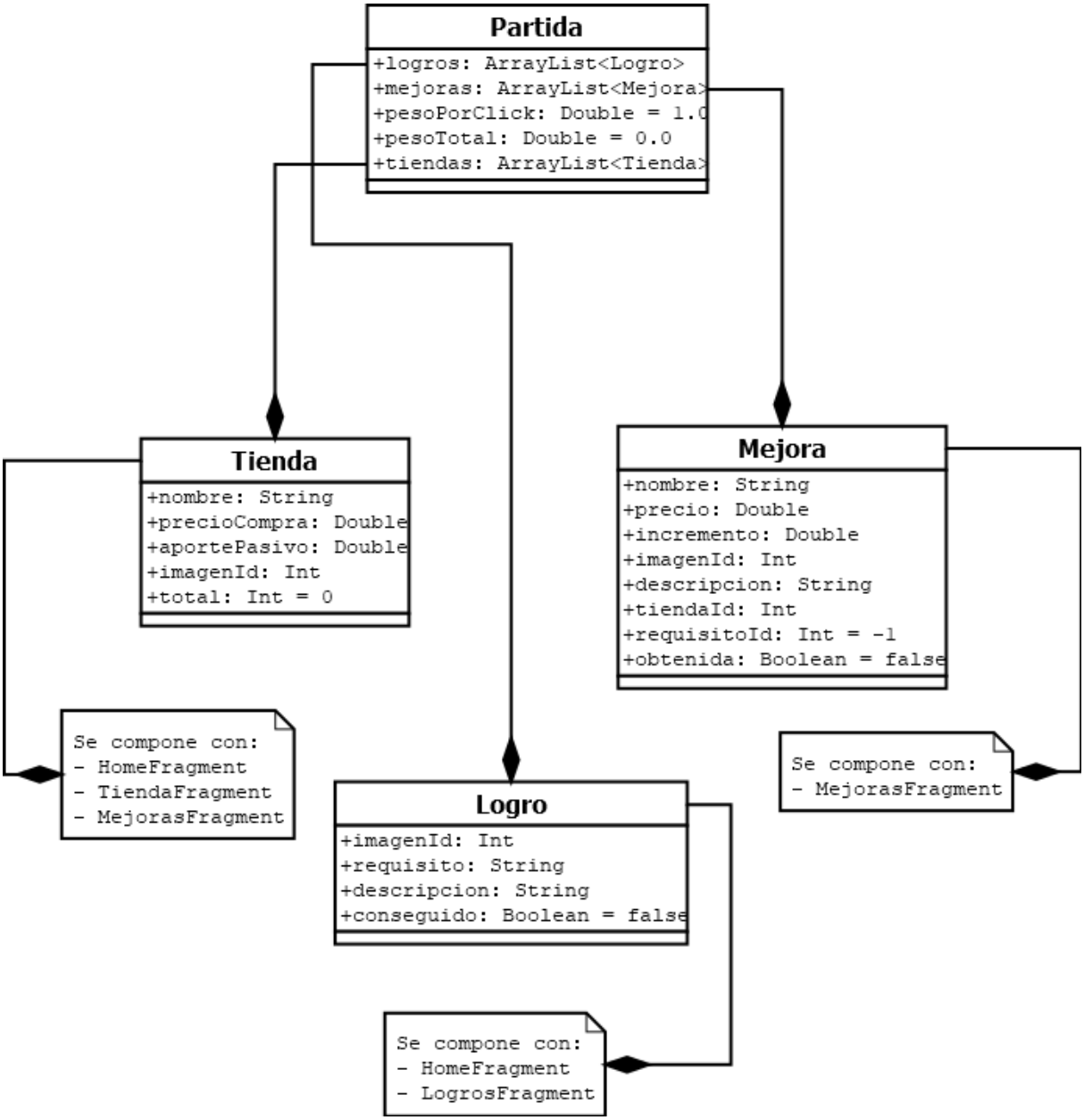
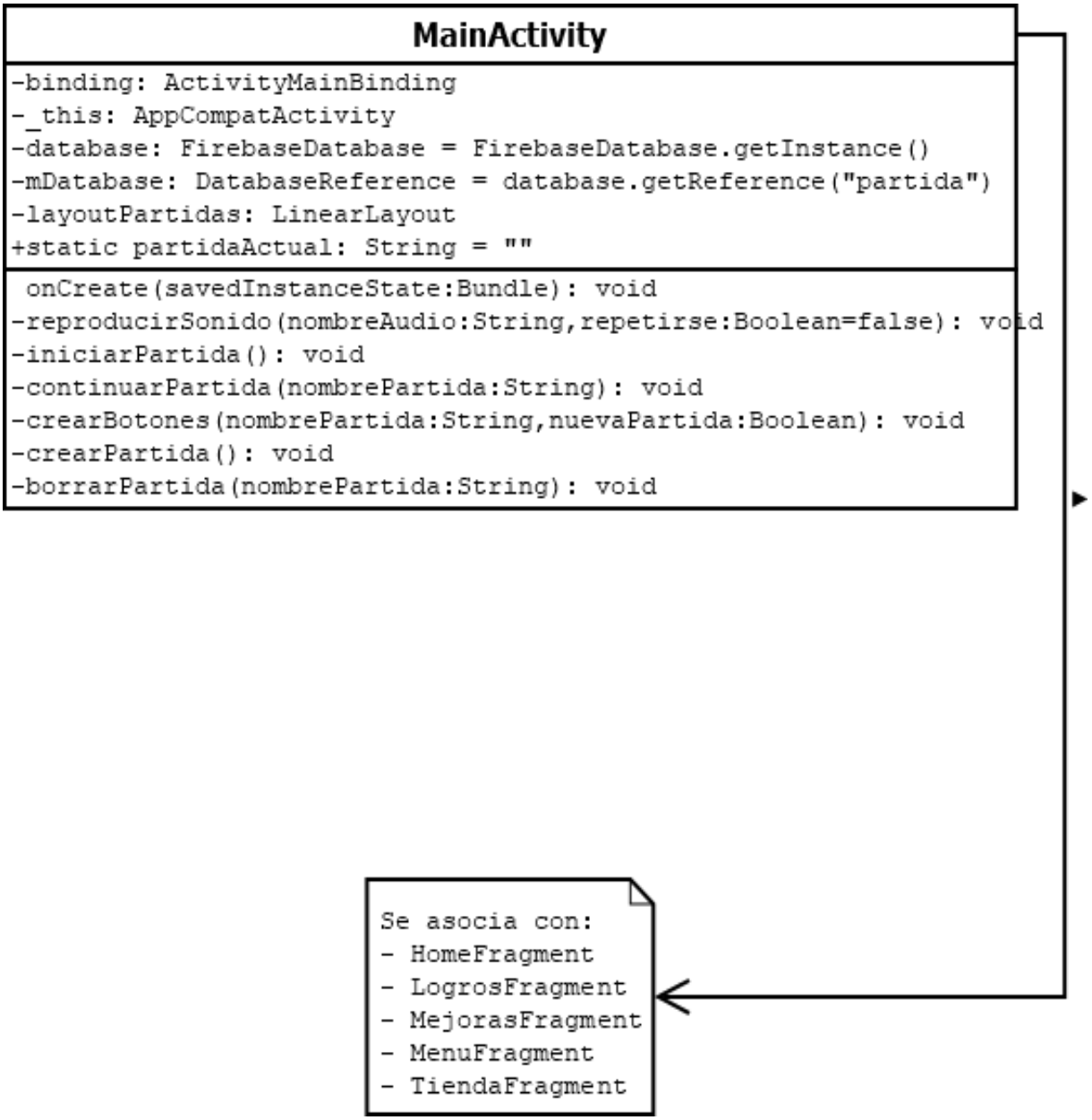
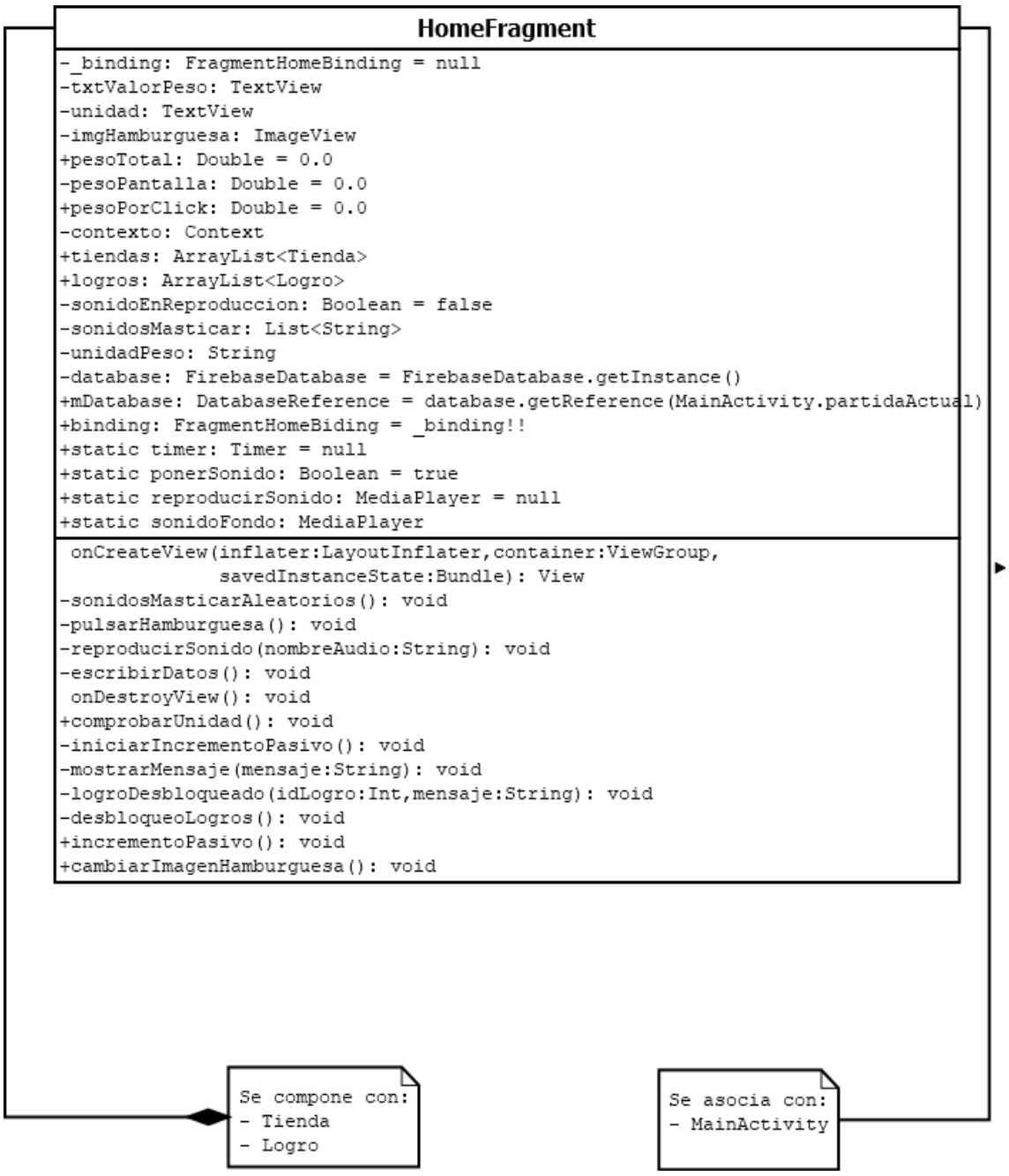
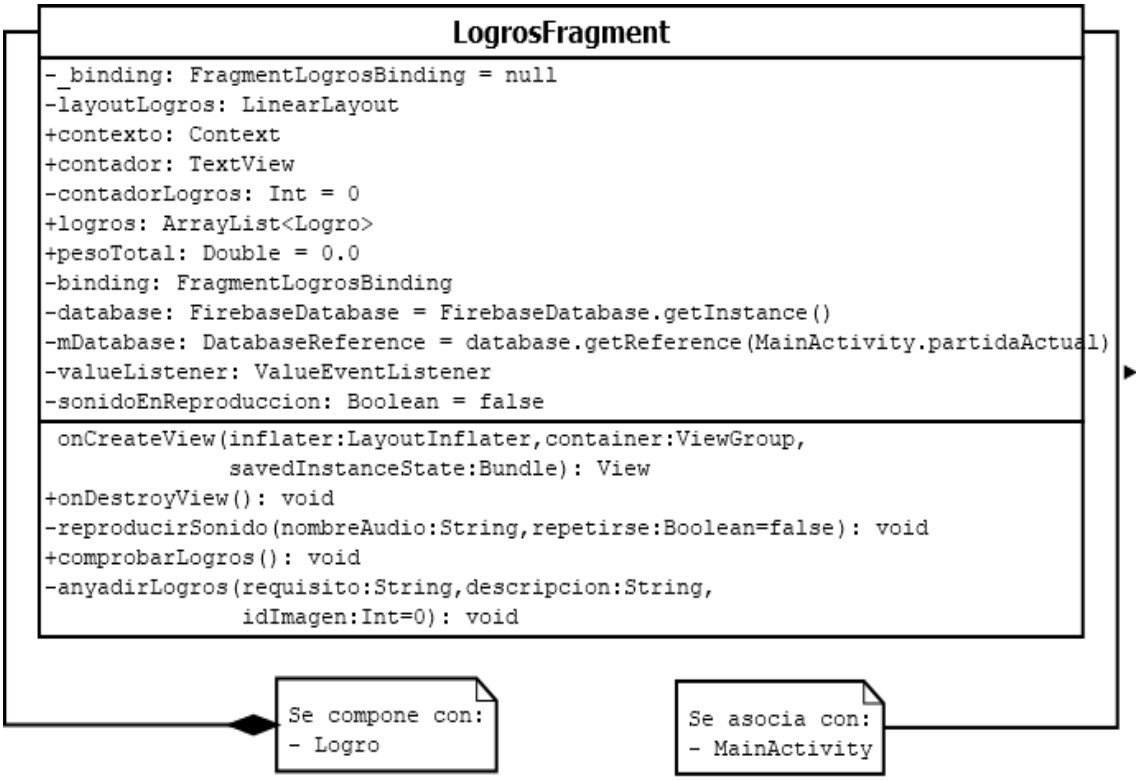


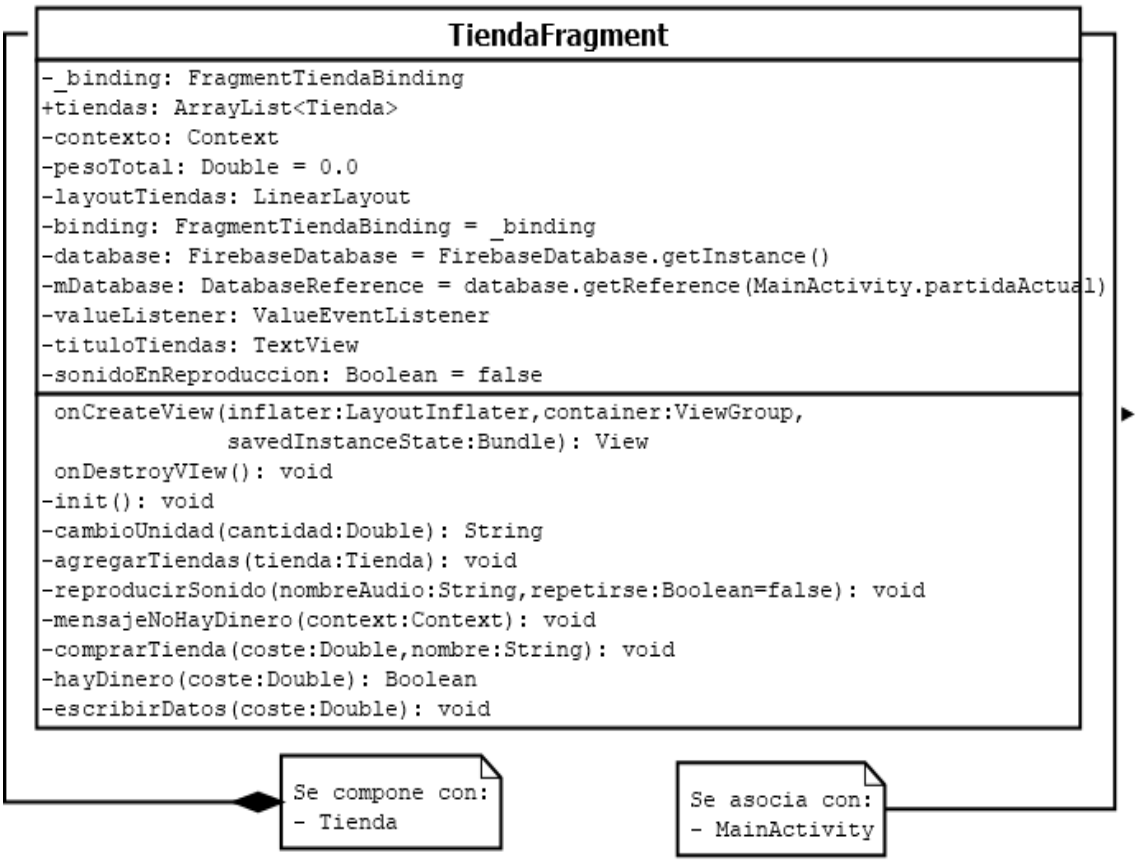
Diagrama de clases

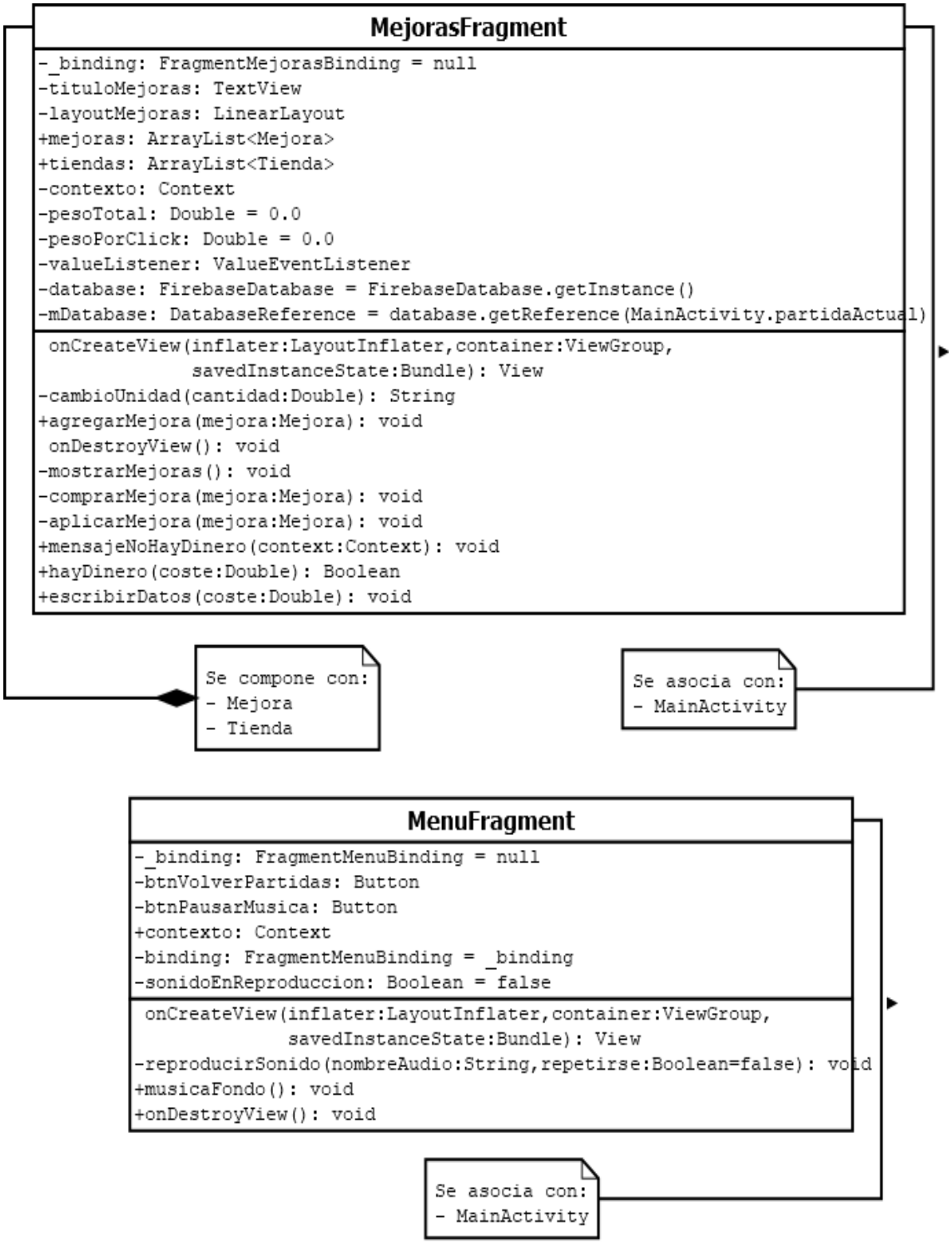



MainActivity
-binding: ActivityMainBinding
-_this: AppCompatActivity
-database: FirebaseDatabase = FirebaseDatabase.getInstance()
-mDatabase: DatabaseReference = database.getReference("partida")
-layoutPartidas: LinearLayout
+static partidaActual: String = ""
onCreate(savedInstanceState:Bundle): void
-reproducirSonido(nombreAudio:String,repetirse:Boolean=false): void
-iniciarPartida(): void
-continuarPartida(nombrePartida:String): void
-crearBotones(nombrePartida:String,nuevaPartida:Boolean): void
-crearPartida(): void
-borrarPartida(nombrePartida:String): void
Se asocia con:
- HomeFragment
- LogrosFragment
- MejorasFragment
- MenuFragment
- TiendaFragment


HomeFragment
-_binding: FragmentHomeBinding = null
-txtValorPeso: TextView
-unidad: TextView
-imgHamburguesa: ImageView
+pesoTotal: Double = 0.0
-pesoPantalla: Double = 0.0
+pesoPorClick: Double = 0.0
-contexto: Context
+tiendas: ArrayList<Tienda>
+logros: ArrayList<Logro>
-sonidoEnReproduccion: Boolean = false
-sonidosMasticar: List<String>
-unidadPeso: String
-database: FirebaseDatabase = FirebaseDatabase.getInstance()
+mDatabase: DatabaseReference = database.getReference(MainActivity.partidaActual)
+binding: FragmentHomeBiding = _binding!!
+static timer: Timer = null
+static ponerSonido: Boolean = true
+static reproducirSonido: MediaPlayer = null
+static sonidoFondo: MediaPlayer
onCreateView(inflater:LayoutInflater,container:ViewGroup,
savedInstanceState:Bundle): View
-sonidosMasticarAleatorios(): void
-pulsarHamburguesa(): void
-reproducirSonido(nombreAudio:String): void
-escribirDatos(): void
onDestroyView(): void
+comprobarUnidad(): void
-iniciarIncrementoPasivo(): void
-mostrarMensaje(mensaje:String): void
-logroDesbloqueado(idLogro:Int,mensaje:String): void
-desbloqueoLogros(): void
+incrementoPasivo(): void
+cambiarImagenHamburguesa(): void
Se compone con:
- Tienda
- Logro
Se asocia con:
- MainActivity


LogrosFragment
-_binding: FragmentLogrosBinding = null
-layoutLogros: LinearLayout
+contexto: Context
+contador: TextView
-contadorLogros: Int = 0
+logros: ArrayList<Logro>
+pesoTotal: Double = 0.0
-binding: FragmentLogrosBinding
-database: FirebaseDatabase = FirebaseDatabase.getInstance()
-mDatabase: DatabaseReference = database.getReference(MainActivity.partidaActual)
-valueListener: ValueEventListener
-sonidoEnReproduccion: Boolean = false
onCreateView(inflater:LayoutInflater,container:ViewGroup,
savedInstanceState:Bundle): View
+onDestroyView(): void
-reproducirSonido(nombreAudio:String,repetirse:Boolean=false): void
+comprobarLogros(): void
-anyadirLogros(requisito:String,descripcion:String,
idImagen:Int=0): void
Se compone con:
- Logro
Se asocia con:
- MainActivity


TiendaFragment
-_binding: FragmentTiendaBinding
+tiendas: ArrayList<Tienda>
-contexto: Context
-pesoTotal: Double = 0.0
-layoutTiendas: LinearLayout
-binding: FragmentTiendaBinding = _binding
-database: FirebaseDatabase = FirebaseDatabase.getInstance()
-mDatabase: DatabaseReference = database.getReference(MainActivity.partidaActual)
-valueListener: ValueEventListener
-tituloTiendas: TextView
-sonidoEnReproduccion: Boolean = false
onCreateView(inflater:LayoutInflater,container:ViewGroup,
savedInstanceState:Bundle): View
onDestroyVIew(): void
-init(): void
-cambioUnidad(cantidad:Double): String
-agregarTiendas(tienda:Tienda): void
-reproducirSonido(nombreAudio:String,repetirse:Boolean=false): void
-mensajeNoHayDinero(context:Context): void
-comprarTienda(coste:Double,nombre:String): void
-hayDinero(coste:Double): Boolean
-escribirDatos(coste:Double): void
Se compone con:
- Tienda
Se asocia con:
- MainActivity


MejorasFragment
-_binding: FragmentMejorasBinding = null
-tituloMejoras: TextView
-layoutMejoras: LinearLayout
+mejoras: ArrayList<Mejora>
+tiendas: ArrayList<Tienda>
-contexto: Context
-pesoTotal: Double = 0.0
-pesoPorClick: Double = 0.0
-valueListener: ValueEventListener
-database: FirebaseDatabase = FirebaseDatabase.getInstance()
-mDatabase: DatabaseReference = database.getReference(MainActivity.partidaActual)
onCreateView(inflater:LayoutInflater,container:ViewGroup,
savedInstanceState:Bundle): View
-cambioUnidad(cantidad:Double): String
+agregarMejora(mejora:Mejora): void
onDestroyView(): void
-mostrarMejoras(): void
-comprarMejora(mejora:Mejora): void
-aplicarMejora(mejora:Mejora): void
+mensajeNoHayDinero(context:Context): void
+hayDinero(coste:Double): Boolean
+escribirDatos(coste:Double): void
Se compone con:
- Mejora
- Tienda
Se asocia con:
- MainActivity
MenuFragment
-_binding: FragmentMenuBinding = null
-btnVolverPartidas: Button
-btnPausarMusica: Button
+contexto: Context
-binding: FragmentMenuBinding = _binding
-sonidoEnReproduccion: Boolean = false
onCreateView(inflater:LayoutInflater,container:ViewGroup,
savedInstanceState:Bundle): View
-reproducirSonido(nombreAudio:String,repetirse:Boolean=false): void
+musicaFondo(): void
+onDestroyView(): void
Se asocia con:
- MainActivity

Requisitos, funcionalidades, tareas, pruebas

R1→ Aplicación contiene vista hamburguesa

R1F1 → Diseño vista hamburguesa.

R1F1T1 → Implementar diseño de la vista.

R1F1T1P1 → Lanzar la aplicación y ver que la vista carga correctamente.

R1F1T1P2 → Comprobar que la vista se vea correctamente en distintos dispositivos.

R1F2 →Poblar diseño con base de datos.

R1F2T1 → Conectarse a la base de datos.

R1F2T2 → Obtener campo peso total de la base de datos.

R1F2T2P1 → Comprobar que se muestra por pantalla el peso de la base.

R1F3 → Incrementar peso total.

R1F3T1→ Al pulsar en la hamburguesa se incrementará el peso total.

R1F3T1P1→Al pulsar en la hamburguesa se muestra por pantalla un incremento de peso.

R1F3T2 → Al incrementar el peso total se actualizará el campo correspondiente en la base de datos.

R1F3T3→ Cambio de unidad del peso por pantalla al llegar a ciertos valores.

R1F3T3P1 → Al llegar a 1000 de la unidad actual se cambiará a la siguiente (por ejemplo, de 1000 miligramos a 1 gramo).

R1F3T3P2 → Al llegar a menos de uno de la unidad actual se cambiará a la anterior (por ejemplo, de 0,95 gramos a 950 miligramos).

R2 → Aplicación contiene vista Tienda.

R2F1. Diseño vista tienda.

R2F1T1 → Implementar el diseño de la vista.

R2F1T1P1 → Lanzar la aplicación y ver que la vista carga correctamente.

R2F1T1P2 → Comprobar que la vista se vea correctamente en distintos dispositivos.

R2F2 → Poblar diseño con base de datos.

R2F2T1 → Conectarse a la base de datos.

R2F2T2 → Obtener campo total tiendas de la base de datos.

R2F2T2P1 → Comprobar que se muestra por pantalla el total de las tiendas de la base de datos.

R2F2T3 → Obtener precio de la tienda de la base de datos.

R2F3 → Comprar tienda

R2F3T1 → Validar el peso suficiente.

R2F3T1P1 → Cerciorarse de que sale un mensaje de alerta de que no se tiene el suficiente peso.

R2F3T1P2 → Comprobar de que no se realiza la compra sin tener el peso.

R2F3T2 → Sumar uno en el total tiendas y que se actualice en la base de datos.

R2F3T3 → Sumar al peso total el incremento pasivo multiplicado por el número de tiendas.

R2F3T3P1 → Ver que se está sumando de forma pasiva la cantidad correcta.

R2F3T4 → Validar que no se pueda comprar una tienda si no se ha comprado ninguna de las tiendas anteriores.

R2F3T4P1 → Comprobar que sale un mensaje de alerta indicando que se compre primero las tiendas anteriores antes de comprar una tienda avanzada

R2F4 → Actualización visual de la hamburguesa.

R2F4T1 → Crear las distintas imágenes de la hamburguesa.

R2F4T2 → Al comprar una nueva tienda la imagen de la hamburguesa cambiara para incorporar el nuevo ingrediente.

R2F4T2P1 → Ver que al comprar una nueva tienda la imagen de la hamburguesa cambia.

R3 → Aplicación contiene vista mejoras.

R3F1 → Diseño vista mejoras.

R3F1T1 → Implementar el diseño de la vista.

R3F1T1P1 → Lanzar la aplicación y ver que la vista carga correctamente.

R3F1T1P2 → Comprobar que la vista se vea correctamente en distintos dispositivos.

R3F2 → Poblar diseño con base de datos.

R3F2T1 → Conectarse a la base de datos.

R3F2T2 → Comprobar que mejoras se deben mostrar

R3F2T2P1 → Comprobar que se muestren las mejoras correctas

R3F2T3 → Comprobar que mejoras ya están desbloqueadas, para mostrar la imagen correspondiente y esconder el botón de compra.

R3F3 → Comprar mejora.

R3F3T1 → Validar el dinero

R3F3T1P1 → Cerciorarse de que sale un mensaje de alerta de que no se tiene el suficiente peso.

R3F3T2 → Cambiar campo de base de datos para indicar que se ha comprado la mejora.

R3F2T2P1 → Comprar una mejora y cerciorarse de que se muestre su imagen, y se esconda su botón de compra.

R3F3T3 → Aplicar la mejora a los clicks o a la tienda correspondiente.

R3F2T3P1 → Comprar una mejora y asegurarse que se aplique.

R4 → Aplicación contiene vista logros.

R4F1 → Diseño vista de logros.

R4F1T1 → Implementar el diseño de la vista.

R4F1T1P1 → Lanzar la aplicación y ver que la vista carga correctamente.

R4F1T1P2 → Comprobar que la vista se vea correctamente en distintos dispositivos.

R4F2 → Poblar diseño con base de datos.

R4F2T1 → Conectarse a la base de datos.

R4F2T2 → Obtener los campos de los logros la base de datos.

R4F2T2P1 → Ver que se muestran correctamente los logros obtenidos.

R4F2T2P2 → Comprobar que los logros tengan sus campos correctos.

R4F3 → Desbloquear logros.

R4F3T1 → Al cumplir una cierta condición se debe desbloquear un logro.

R4F3T2 → Al desbloquear un logro se actualizará su campo en la base de datos para indicar que se ha obtenido.

R4F4 → Contador de logros.

R4F4T1 → Crear un contador que incremente a medida que se vayan desbloqueando los logros.

R4F4T1P1 → Cerciorarse de que el contador muestra el resultado esperado al desbloquear logros.

R4F5 → Alerta al desbloquear logros.

R4F5T1 → Crear un mensaje de alerta para cada uno de los logros.

R4F5T2 → Mostrar el mensaje cuando se desbloquee dicho logro.

R4F5T2P1 → Comprobar de que al desbloquear un logro se muestra su alerta correspondiente.

R4F5T2P2 → Ver que no se repite el mensaje de alerta más de una vez.

R5 → Aplicación contiene vista menú.

R5F1 → Diseño vista de menú.

R5F1T1 → Implementar el diseño de la vista.

R5F1T1P1 → Lanzar la aplicación y ver que la vista carga correctamente.

R5F1T1P2 → Comprobar que la vista se vea correctamente en distintos dispositivos.

R5F2 → Botón regreso vista partidas.

R5F2T1 → Añadir un botón que al pulsarlo te retorne a la vista de partidas, que pausa la música de fondo.

R5F2T1P1 → Ver que al pulsarlo te lleve a la vista partida y no a otra.

R5F3 → Botón de pausar o reanudar la música de fondo.

R5F3T1 → Añadir un botón que al pulsarlo se pause o reanude la música de fondo.

R5F3T1P1 → Comprobar que la música de fondo se pausa y reanuda sin problemas.

R6 → Aplicación contiene la vista partidas.

R6F1 → Diseño vista de partidas.

R6F1T1 → Implementar el diseño de la vista.

R6F1T1P1 → Lanzar la aplicación y ver que la vista carga correctamente.

R6F1T1P2 → Comprobar que la vista se vea correctamente en distintos dispositivos.

R6F1T2 → Implementar tres botones que permitan crear partidas.

R6F2 → Poblar diseño con base de datos.

R6F2T1 → Conectarse a la base de datos.

R6F2T2 → Obtener las partidas existentes de la base de datos.

R6F2T2P1 → Ver que se muestran correctamente las partidas.

R6F2T2P2 → Comprobar que las partidas tengan sus datos correctos.

R6F3 → Crear partida.

R6F3T1 → Crear la partida cuando el usuario pulse el botón correspondiente, hasta un máximo de tres partidas.

R6F3T1P1 → Cerciorarse de que la partida se crea y esta se muestre por pantalla.

R6F3T2 → El usuario puede introducir el nombre de su elección.

R6F3T2P1 → Comprobar que la partida se cree con el nombre correspondiente.

R6F4 → Borrar partida.

R6F4T1 → Al crear una partida aparecerá colindando a ella un botón que permita borrar dicha partida.

R6F4T1P1 → Confirmar que aparezca el botón de borrar a crear una partida.

R6F4T2 → Al pulsar el botón asociado a una partida esta será borrada.

R6F4T2P1 → Comprobar que al pulsar el botón de borrar la partida se borra.

R6F4T2P2 → Ver que el botón con el nombre de la partida vuelve a ser el botón de crear partida y se elimina el botón de borrar.

R7 → Implementación de sonido.

R7F1 → Música de fondo.

R7F1T1 → Editar la música para que se adapte a la aplicación.

R7F1T2 → Crear una función que reproduzca la música de fondo en toda la aplicación.

R7F1T2P1 → Comprobar que la música se reproduce de manera correcta en cualquier parte de la aplicación.

R7F2 → Sonidos al entrar en cada sección.

R7F2T1 → Editar los sonidos para que se adapten a la aplicación.

R7F2T2 → Cada sección tendrá asociado un sonido relacionado con la función que desempeña.

R7F2T2P1 → Comprobar que se escuchan los sonidos correspondientes de cada sección.

R7F3 → Sonido al pulsar la hamburguesa.

R7F3T1 → Editar los sonidos para que se adapten a la aplicación.

R7F3T2 → Crear una lista de sonidos aleatorios.

R7F3T3 → Asociar la lista aleatoria con el evento de pulsar en la hamburguesa.

R7F3T3P1 → Comprobar que todos los sonidos se escuchan al pulsar la hamburguesa.

R7F3T4 → Hacer que no se solapen los distintos sonidos.

R7F3T4P1 → Al pulsar no se reproducirá ningún otro sonido hasta que el actual no haya terminado.

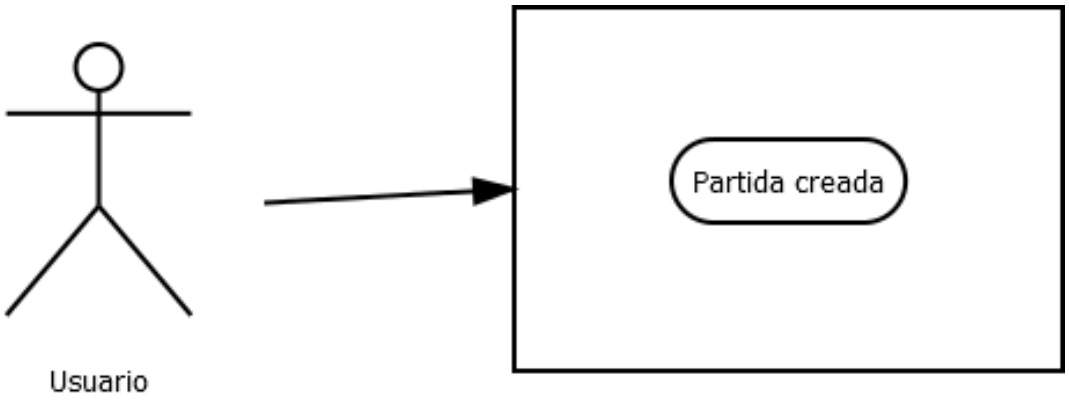
R7F4 → Sonido de desbloqueo de logro.

R7F4T1 → Editar el sonido para que se adapte a la aplicación.

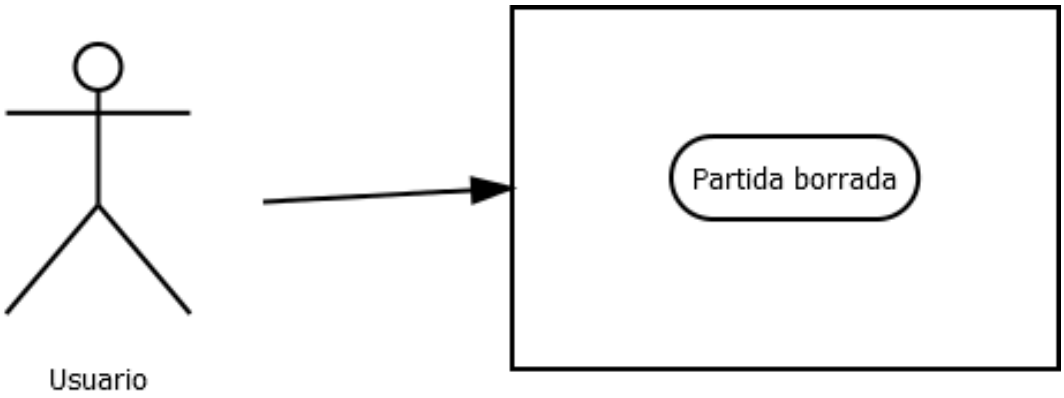
R7F4T2 → Hacer que se reproduzca el sonido junto con el mensaje de alerta al desbloquear un logro.

R7F4T2P1 → Comprobar que se escucha el sonido al desbloquear un logro.

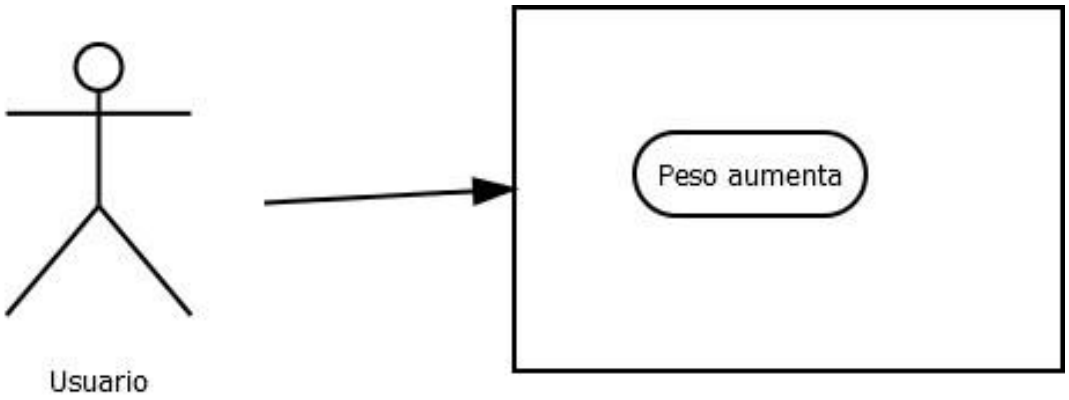
Casos de uso



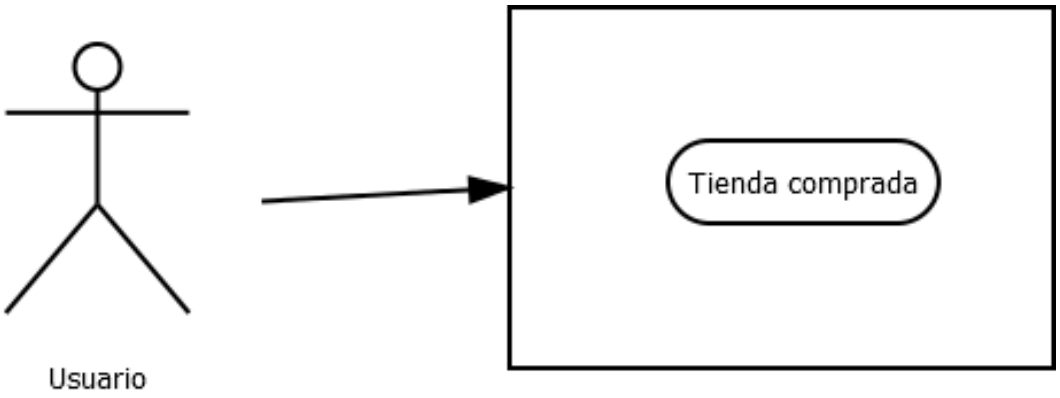
Nombre del caso de uso	CU01
Actor(es)	Usuario
Descripción	Al pulsar en el botón de crear una nueva partida, el usuario escogerá el nombre de su elección.
Evento desencadenado	Se crea una partida nueva con el nombre introducido por el usuario y al lado se crea un botón para poder borrarla.
Pasos realizados	
1. Pulsar el botón "crear nueva partida".	
2. Introducir un nombre para dicha partida.	
3. Se almacena la nueva partida en la base de datos.	
4. El botón "crear una nueva partida" pasa a ser un botón con la partida creada por el usuario.	
Precondiciones	Ninguna.
Postcondiciones	Poder jugar.
Vista	Activity_main.xml
Tabla	partida
Clase	MainActivity.kt



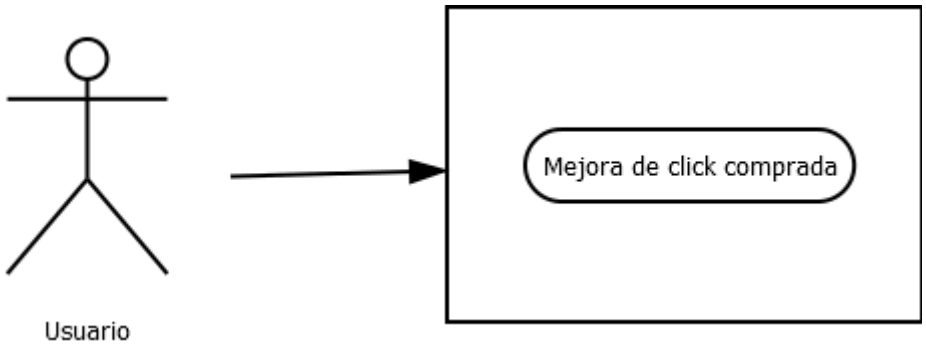
Nombre del caso de uso	CU02
Actor(es)	Usuario
Descripción	Al pulsar en el botón de borrar partida, la partida se eliminará.
Evento desencadenado	La partida se borrará de la base de datos y el botón volverá a ser el de “crear una nueva partida”.
Pasos realizados	
1. Pulsar el botón para borrar una partida.	
2. La partida se borra en la base de datos.	
3. El botón de la partida cambia al estado previo de “crear una nueva partida”.	
Precondiciones	Haber creado una partida.
Postcondiciones	La partida se elimina.
Vista	Activity_main.xml
Tabla	partida
Clase	MainActivity.kt



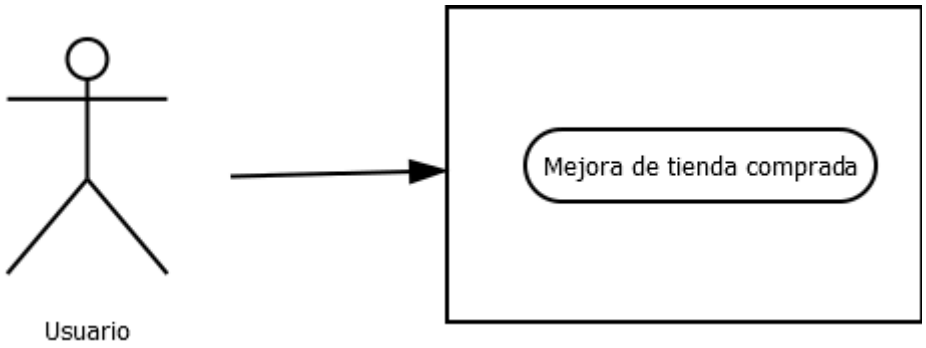
Nombre del caso de uso	CU03
Actor(es)	Usuario
Descripción	El usuario al pulsar en la hamburguesa incrementa el peso total de su partida y almacena el nuevo valor en la base de datos
Evento desencadenado	Al pulsar se incrementa el peso total en la base de datos además de el que se muestra por pantalla.
Pasos realizados	
1. Pulsar la hamburguesa	
2. Incrementa el peso total	
3. Se almacena el nuevo valor en la base de datos	
4. Se muestra por pantalla el valor actualizado	
Precondiciones	Haber seleccionado una partida
Postcondiciones	El peso total se incrementa en 1
Vista	Fragment_home.xml
Tabla	partida / pesoTotal
Clase	MainActivity.kt / HomeFragment.kt



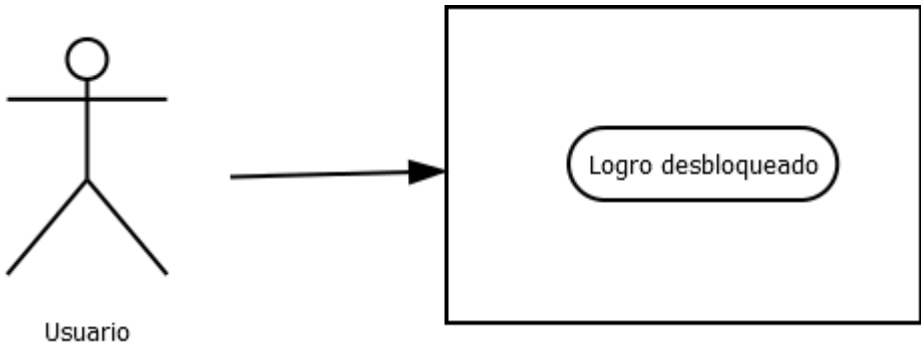
Nombre del caso de uso	CU04
Actor(es)	Usuario
Descripción	El usuario al pulsar en el botón “comprar tienda” adquirirá la tienda.
Evento desencadenado	Se incrementará en uno el total de la tienda que se haya comprado y proporcionará el peso de forma pasiva asociado a la tienda.
Pasos realizados	
1. Pulsar el botón comprar asociado a una tienda.	
2. Resta el valor al peso total.	
3. Se incrementa en uno el contador del total de la tienda.	
4. Se muestra por pantalla el contador actualizado.	
5. Se actualiza el aporte pasivo de peso de dicha tienda.	
Precondiciones	Tener el peso suficiente.
Postcondiciones	Aumentan los ingresos pasivos.
Vista	Fragment_tienda.xml
Tabla	partida / tiendas
Clase	MainActivity.kt / TiendaFragment.kt



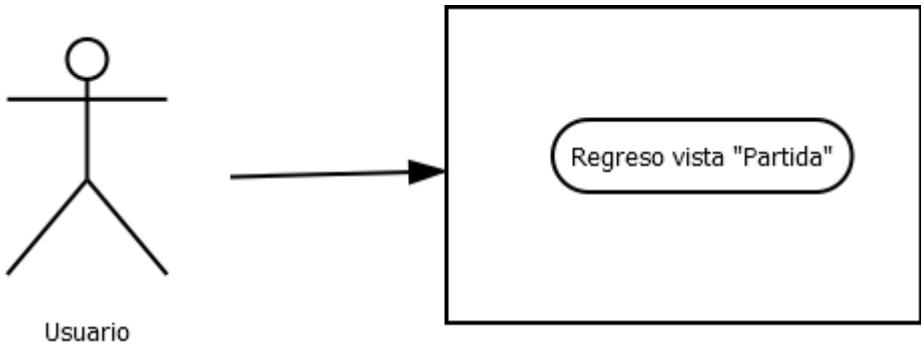
Nombre del caso de uso	CU05
Actor(es)	Usuario
Descripción	El usuario adquiere una mejora de click.
Evento desencadenado	La mejora otorgará al usuario un aumento en el peso que se obtiene por click.
Pasos realizados	
1. Pulsar el botón “Comprar” asociado a una mejora de click.	
2. Se resta al peso total el valor de la mejora comprada.	
3. La mejora adquirida empieza a surtir efecto sobre los clicks en la partida actual.	
Precondiciones	Tener el peso suficiente.
Postcondiciones	Aumenta el peso obtenido por click.
Vista	Fragment_tiendas.xml / fragment_mejoras.xml
Tabla	partida / mejoras
Clase	MainActivity.kt / MejoraFragment.kt



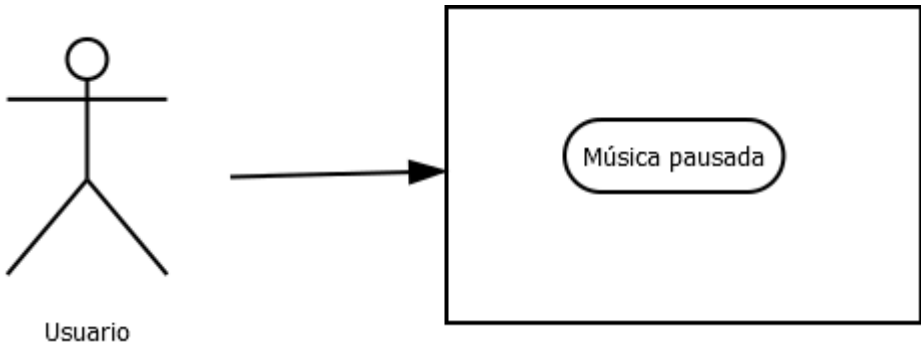
Nombre del caso de uso	CU06
Actor(es)	Usuario
Descripción	El usuario adquiere una mejora de tienda.
Evento desencadenado	La mejora incrementará el peso por segundo que aporta la tienda en cuestión.
Pasos realizados	
1. Pulsar el botón “Comprar” asociado a una mejora de una tienda.	
2. Se resta al peso total el valor de la mejora comprada.	
3. La mejora adquirida empieza a surtir efecto sobre el peso pasivo que aporta la tienda en la partida actual.	
Precondiciones	Tener el peso suficiente.
Postcondiciones	Implementa la funcionalidad de la mejora.
Vista	Fragment_tiendas.xml / fragment_mejoras.xml
Tabla	partida / mejoras
Clase	MainActivity.kt / MejoraFragment.kt



Nombre del caso de uso	CU07
Actor(es)	Usuario
Descripción	Al ir progresando la partida el usuario desbloqueara distintos logros, que aparecerán en la sección “Logros”.
Evento desencadenado	Al conseguir cierto requisito saldrá por pantalla un mensaje emergente junto con un sonido desvelando un nuevo logro desbloqueado.
Pasos realizados	
1. Alcanzar el requisito de cada uno de los logros.	
2. Se mostrará por pantalla el logro desbloqueado con un mensaje.	
3. Junto al mensaje se reproducirá un sonido para advertir del logro desbloqueado.	
4. Se mostrará en la sección “Logros” el logro desbloqueado”.	
5. Se incrementará el contador de logros totales de la sección “Logros”.	
Precondiciones	Ninguna
Postcondiciones	Logro desbloqueado.
Vista	Fragment_logros.xml
Tabla	partida / logros
Clase	MainActivity.kt / LogrosFragment.kt



Nombre del caso de uso	CU08
Actor(es)	Usuario
Descripción	El usuario al pulsar en el botón “Volver a la lista de partidas” regresara a la vista de partidas.
Evento desencadenado	Al pulsarlo el usuario regresará a la lista de las partidas y la música de fondo se pausará.
Pasos realizados	
1. Pulsar el botón “Volver a la lista de partidas” de la sección menú.	
2. Se pausa la música de fondo.	
3. Se paran los incrementos pasivos.	
4. Se cambia a la vista “Partidas”.	
Precondiciones	Haber seleccionado una partida.
Postcondiciones	La vista cambiara a la de “Partidas”.
Vista	Fragment_menu.xml
Tabla	partida
Clase	MainActivity.kt / MenuFragment.kt



Nombre del caso de uso	CU09
Actor(es)	Usuario
Descripción	El usuario al pulsar en el botón “Pausar la música” la música se pausará.
Evento desencadenado	Al pulsarlo la música de fondo dejará de sonar y el texto del botón cambiará a “Reanudar la música”.
Pasos realizados	
1. Pulsar el botón “Pausar la música” de la sección menú.	
2. Se pausa la música de fondo.	
3. El texto del botón cambia a “Reanudar la música”.	
Precondiciones	Haber seleccionado una partida.
Postcondiciones	La música de fondo se para.
Vista	Fragment_menu.xml
Tabla	Ninguna
Clase	MainActivity.kt / MenuFragment.kt

Pruebas

Nombre	***Descripción***	***Completado***
R1F1T2P1, R1F1T1P2, R2F1T1P1, R2F1T1P2, R3F1T1P1, R3F1T1P2, R4F1T1P1, R4F1T1P2, R5F1T1P1, R5F1T1P2, R6F1T1P1, R6F1T1P2	Iniciar la aplicación y comprobar que carguen todas las vistas correctamente.	Al iniciar la aplicación se ha comprobado que las vistas se han cargado y generado correctamente.
R1F2T2P1, R2FT2P1, R3F2T2P1, R4F2T2P1, R4F2T2P2, R6F2T2P1, R6F2T2P2	Lanzar la aplicación y comprobar que se están obteniendo correctamente los datos de la base de datos.	Se ha lanzado la aplicación, con algunos datos almacenados en la BD y se ha verificado que se muestran correctamente.
R1F3T1P1	Desde la vista hamburguesa, pulsar en la hamburguesa	Al pulsar en la hamburguesa el peso que se muestra por pantalla se va incrementando.
R1F3T3P1, R1F3T3P2	Desde la vista hamburguesa, comprobar que, al cambiar el peso desde BD, se cambia correctamente la unidad, tanto al sumar como al restar.	Al cambiar el peso desde BD, se ha comprobado que la unidad de medida actual va cambiando correctamente al aumentar o disminuir el peso total.
R2F3T1P1, R2F3T1P2	En la vista tiendas intentar comprar una nueva tienda sin tener el peso suficiente.	Se ha comprobado que salta el aviso de que no hay peso suficiente

R2F3T3P1	Obtener una tienda y comprobar que otorga el precio pasivo correcto.	Al comprarse una tienda se ha podido verificar que genera peso de forma pasiva.
R2F3T4P1	Intentar comprar una tienda sin comprar las tiendas previas.	No se ha llegado a implementar en la versión actual de la aplicación.
R2F4T2P1	Comprar una nueva tienda y verificar que cambie la imagen de la hamburguesa para incorporar el nuevo ingrediente.	Al comprar una nueva tienda se actualiza la imagen de la hamburguesa para incorporar el nuevo ingrediente comprado.
R3F3T1P1	Intentar comprar una mejora sin tener el peso necesario.	Al comprar una nueva mejora sin tener el peso suficiente salta el mensaje de alerta.
R3F2T2P1, R3F2T3P1	Comprar una nueva mejora teniendo el peso necesario.	Al comprar una mejora teniendo el peso requerido, esta se aplica correctamente
R4F4T1P1	Comprobar que el contador de logros muestre correctamente el total de logros obtenidos.	Si se han desbloqueado algunos logros, el contador muestra correctamente el total de logros desbloqueados
R4F5T2P1, R4F5T2P2	Al desbloquear un logro debe mostrarse una alerta una única vez.	Al desbloquear un logro la alerta se muestra correctamente una única vez.

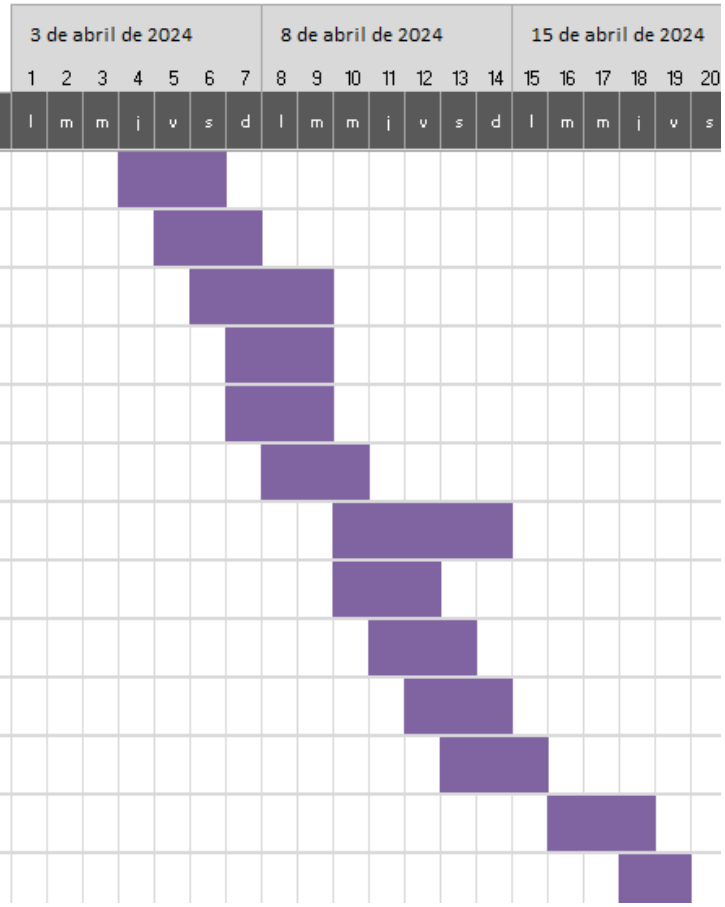
R5F2T1P1	Pulsar el botón de retorno a la vista partidas.	El botón regresa correctamente a la vista partidas.
R5F3T1P1	Pulsar el botón de pausar música de fondo.	El botón pausa correctamente la música.
R6F3T1P1, R6F3T2P1	Crear una nueva partida con un nombre cualquiera.	Se puede crear una partida con el nombre que elija el usuario.
R6F4T1P1, R6F4T2P1, R6F4T2P2	Verificar que el botón de borrar partida aparezca cuando le corresponde, y que funcione correctamente.	Al crear una nueva partida, aparece el botón para borrarla y al pulsarlo dicha partida se elimina.
R7F1T2P1	Verificar el funcionamiento de la música de fondo.	Al entrar en la aplicación la música se escucha correctamente.
R7F2T2P1	Comprobar que se escuchen los sonidos de cada sección de la aplicación.	Al entrar en una nueva sección el sonido se carga correctamente.
R7F3T3P1, R7F3T4P1	Pulsar en la hamburguesa y comprobar que se escuchen los distintos sonidos sin interrumpirse entre sí y de forma aleatoria.	Al pulsar en la hamburguesa se reproduce un sonido aleatorio de la lista, sin interrumpirse entre si.
R7F4T2P1	Desbloquear un logro y verificar que suene el sonido correspondiente.	Al desbloquear un logro se escucha el sonido.


Diagrama de Gantt
Inicio del proyecto:
Semana para mostrar:
1
20 de marzo de 2024
25 de marzo de 2024
1 de abril de 2024
18 19 20 21 22 23 24 25 26 27 28 29 30 31 1 2 3 4 5 6 7
l m m j v s d l m m j v s d
TAREA
HORAS
ASIGNADO A
INICIO
FIN
R1F1T1 2h Javier 20-3-24 21-3-24
R1F2T1 30 min Javier 21-3-24 21-3-24
R1F2T2 30 min Javier / Andres 22-3-24 22-3-24
R1F3T1 30 min Andres 22-3-24 23-3-24
R1F3T2 1h Javier / Andres 24-3-24 26-3-24
R1F3T3 2h Javier / Andres 26-3-24 27-3-24
R2F1T1 4h Javier 28-3-24 31-3-24
R2F2T1 30 min Javier 31-3-24 31-3-24
R2F2T2 30 min Andres 1-4-24 2-4-24
R2F2T3 1h Andres 1-4-24 3-4-24
R2F3T1 30 min Andres 1-4-24 3-4-24
R2F3T2 1h Javier / Andres 4-4-24 6-4-24

Inicio del proyecto: mi, 3/20/2024

Semana para mostrar: 3

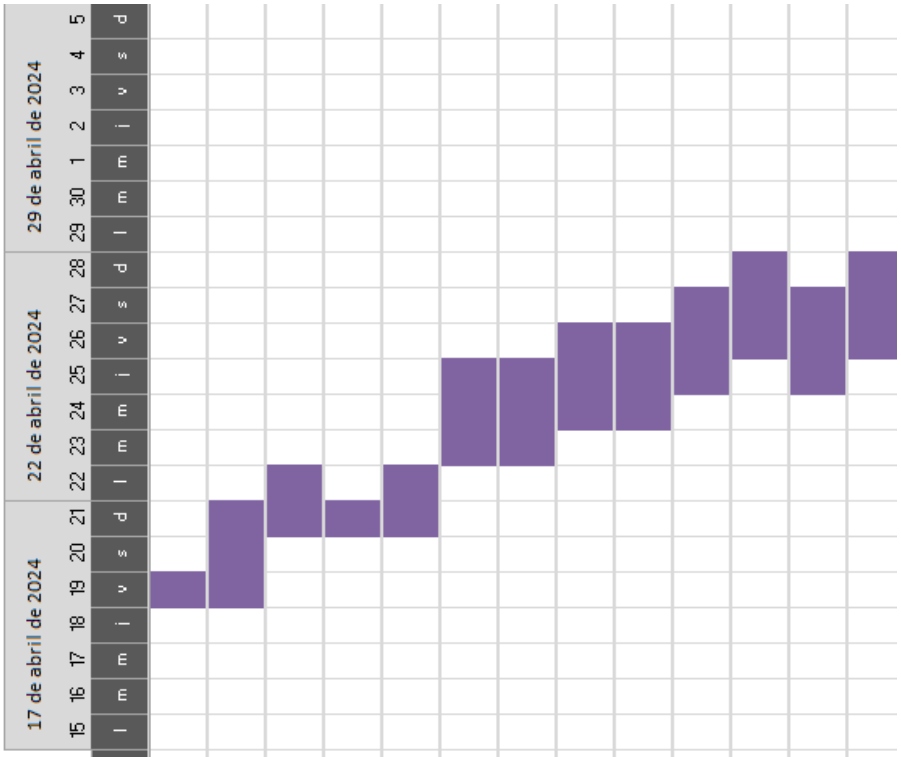
TAREA	HORAS	ASIGNADO A	INICIO	FIN
R2F3T3	1.50h	Andres	4-4-24	6-4-24
R2F3T4	1h	Andres	5-4-24	7-4-24
R2F4T1	2h	Javier	6-4-24	9-4-24
R2F4T2	1h	Javier	7-4-24	9-4-24
R3F1T1	2h	Andres	7-4-24	9-4-24
R3F2T1	30 min	Andres	8-4-24	10-4-24
R3F2T2	4h	Andres	10-4-24	14-4-24
R3F2T3	1h	Andres	10-4-24	12-4-24
R3F3T1	30 min	Andres	11-4-24	13-4-24
R3F3T2	1h	Andres	12-4-24	14-4-24
R3F3T3	4	Andres	13-4-24	15-4-24
R4F1T1	2.5h	Javier	16-4-24	18-4-24
R4F2T1	30 min	Javier	18-4-24	19-4-24



Inicio del proyecto: mi, 3/20/2024

Semana para mostrar: 5

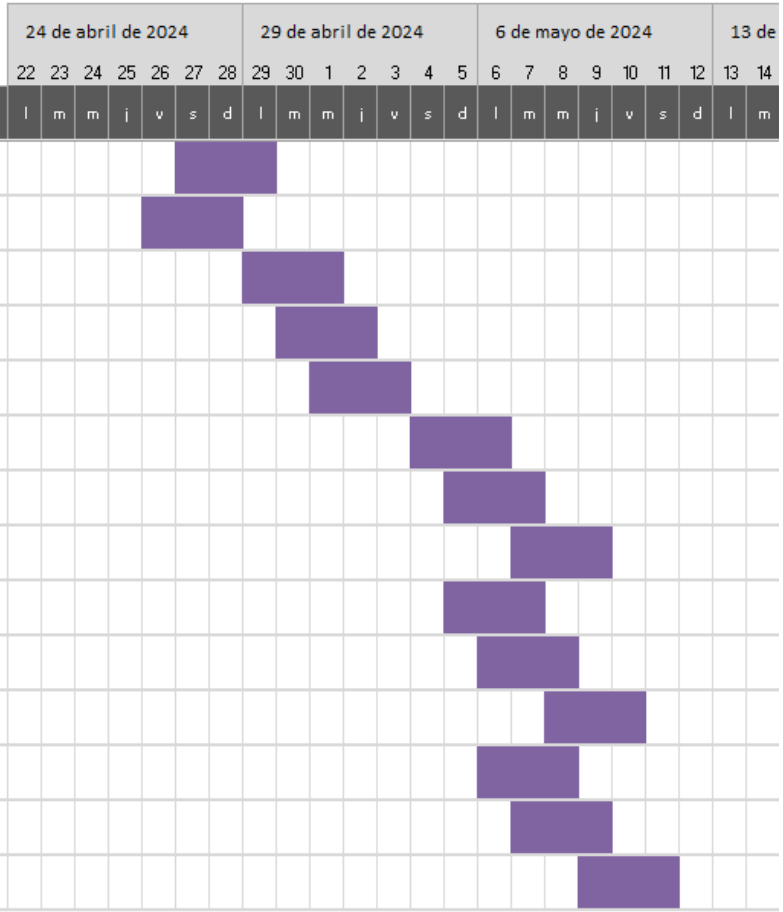
TAREA	HORAS	ASIGNADO A	INICIO	FIN
R4F2T2	30 min	Javier / Andres	19-4-24	19-4-24
R4F3T1	2.50h / 2h	Javier / Andres	19-4-24	21-4-24
R4F3T2	1h / 30 min	Javier / Andres	21-4-24	22-4-24
R4F4T1	30 min	Javier	21-4-24	21-4-24
R4F5T1	1h	Javier	21-4-24	22-4-24
R4F5T2	1h / 2h	Javier / Andres	23-4-24	25-4-24
R5F1T1	1h	Javier	23-4-24	25-4-24
R5F2T1	30 min	Javier	24-4-24	26-4-24
R5F3T1	30 min	Javier	24-4-24	26-4-24
R6F1T1	30 min	Javier	25-4-24	27-4-24
R6F1T2	30 min	Andres	26-4-24	28-4-24
R6F2T1	30 min	Javier	25-4-24	27-4-24
R6F2T2	30 min	Andres	26-4-24	28-4-24



Inicio del proyecto: mi, 3/20/2024

Semana para mostrar: 6

TAREA	HORAS	ASIGNADO A	INICIO	FIN
R6F3T1	1h	Andres	27-4-24	29-4-24
R6F3T2	30 min	Andres	26-4-24	28-4-24
R6F4T1	30 min	Andres	29-4-24	1-5-24
R6F4T2	1h	Andres	30-4-24	2-5-24
R7F1T1	30 min	Javier	1-5-24	3-5-24
R7F1T2	30 min	Javier	4-5-24	6-5-24
R7F2T1	30 min	Javier	5-5-24	7-5-24
R7F2T2	30 min	Javier	7-5-24	9-5-24
R7F3T1	30 min	Javier	5-5-24	7-5-24
R7F3T2	15 min	Javier	6-5-24	8-5-24
R7F3T3	15 min	Javier	8-5-24	10-5-24
R7F3T4	1h	Andres	6-5-24	8-5-24
R7F4T1	15 min	Javier	7-5-24	9-5-24
R7F4T2	30 min	Javier	9-5-24	11-5-24
TOTAL DE HORAS	JAVIER 30.75h	ANDRES 31.5h		



PLANES A FUTURO

Se ha conseguido implementar la gran mayoría de funcionalidades que se habían propuesto para la aplicación, sin embargo, hay algunas que no se han desarrollado del todo o pueden incluir mejoras:

- Partidas vinculadas a la cuenta de Google Play de cada usuario, para que pueda jugar desde cualquier dispositivo Android.
- Logros adicionales, posiblemente relacionados con las mejoras, u otros aspectos del juego.
- Mejorar la adquisición de peso pasivo y por clic, para que sea más coherente y garantice un progreso más constante.
- Mejorar los precios de las tiendas y las mejoras, buscar que sigan alguna fórmula incremental.
- Eventos aleatorios, que beneficien a usuarios más atentos e involucrados.
- Traducir el juego a otros idiomas, principalmente al inglés.
- Mejorar el rendimiento de la aplicación.
- No dejar que el usuario compre una tienda sin haber desbloqueado las anteriores.

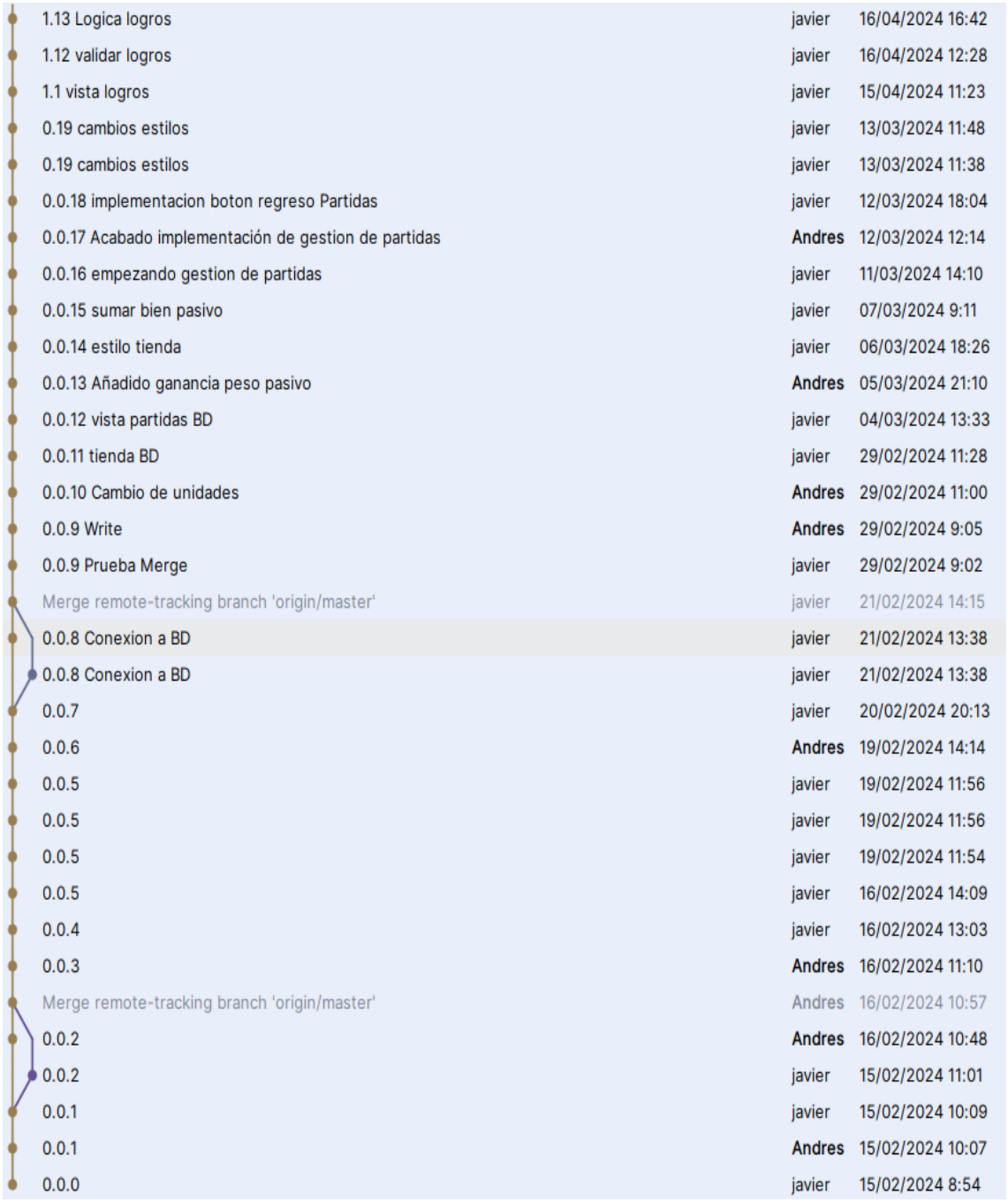
GIT

Enlace al proyecto

https://github.com/Javierpc14/BurguerClicker

Capturas





1.13 Logica logros	javier	16/04/2024 16:42
1.12 validar logros	javier	16/04/2024 12:28
1.1 vista logros	javier	15/04/2024 11:23
0.19 cambios estilos	javier	13/03/2024 11:48
0.19 cambios estilos	javier	13/03/2024 11:38
0.0.18 implementacion boton regreso Partidas	javier	12/03/2024 18:04
0.0.17 Acabado implementación de gestion de partidas	Andres	12/03/2024 12:14
0.0.16 empezando gestion de partidas	javier	11/03/2024 14:10
0.0.15 sumar bien pasivo	javier	07/03/2024 9:11
0.0.14 estilo tienda	javier	06/03/2024 18:26
0.0.13 Añadido ganancia peso pasivo	Andres	05/03/2024 21:10
0.0.12 vista partidas BD	javier	04/03/2024 13:33
0.0.11 tienda BD	javier	29/02/2024 11:28
0.0.10 Cambio de unidades	Andres	29/02/2024 11:00
0.0.9 Write	Andres	29/02/2024 9:05
0.0.9 Prueba Merge	javier	29/02/2024 9:02
Merge remote-tracking branch 'origin/master'	javier	21/02/2024 14:15
0.0.8 Conexion a BD	javier	21/02/2024 13:38
0.0.8 Conexion a BD	javier	21/02/2024 13:38
0.0.7	javier	20/02/2024 20:13
0.0.6	Andres	19/02/2024 14:14
0.0.5	javier	19/02/2024 11:56
0.0.5	javier	19/02/2024 11:56
0.0.5	javier	19/02/2024 11:54
0.0.5	javier	16/02/2024 14:09
0.0.4	javier	16/02/2024 13:03
0.0.3	Andres	16/02/2024 11:10
Merge remote-tracking branch 'origin/master'	Andres	16/02/2024 10:57
0.0.2	Andres	16/02/2024 10:48
0.0.2	javier	15/02/2024 11:01
0.0.1	javier	15/02/2024 10:09
0.0.1	Andres	15/02/2024 10:07
0.0.0	javier	15/02/2024 8:54

BIBLIOGRAFIA

FireBase. (s.f.). Obtenido de https://firebase.google.com/

GitHub. (s.f.). Obtenido de https://github.com

Icon Library. (s.f.). Obtenido de https://icon-library.com/

JSONCrack. (s.f.). Obtenido de https://jsoncrack.com/editor

King, A. (22 de 5 de 2015). *tutsplus*. Obtenido de https://code.tutsplus.com/numbers-getting-bigger-what-are-incremental-games-and-why-are-they-fun--cms-23925a

Lucidspark. (s.f.). Obtenido de https://lucid.app/lucidspark/c6d94c79-0439-4677-bdf0-2c89dac6d645/edit?invitationId=inv_167e0f73-237a-4364-ad05-e3245bc90838&page=0_0#

Optimizilla. (s.f.). Obtenido de https://imagecompressor.com/

Photopea. (s.f.). *Photopea*. Obtenido de https://www.photopea.com/

PixaBay. (s.f.). Obtenido de https://pixabay.com/es/

soundbuttonsworld. (s.f.). *Sound Buttons*. Obtenido de https://soundbuttonsworld.com/

Team, C. (10 de 05 de 2021). *Codecademy*. Obtenido de https://www.codecademy.com/resources/blog/what-is-kotlin-used-for/

Vecteezy. (s.f.). Obtenido de https://es.vecteezy.com/